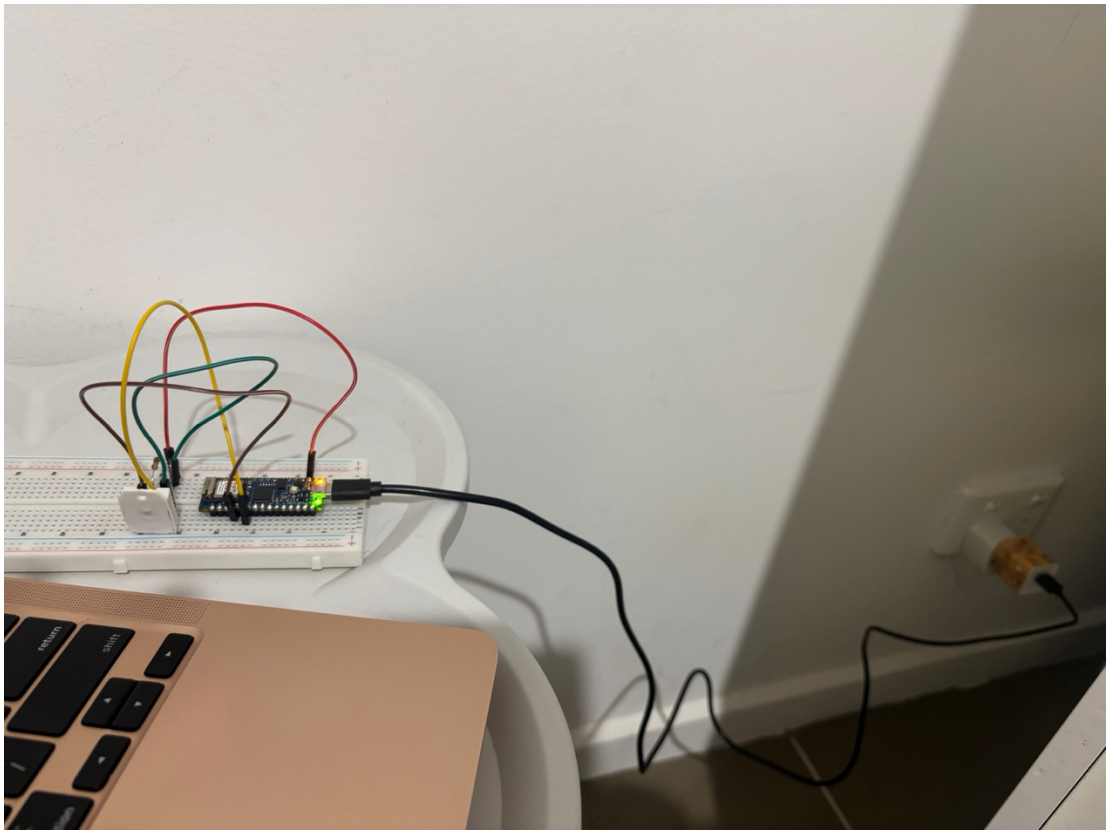
# Part 1: Protocol execution and setup

## 1.1 Hardware and software setup

One DHT22 temperature and humidity sensor on an Arduino Nano 33 IoT, powered from a USB phone charger, took a reading every 30 seconds and sent it over WiFi to a Firebase Realtime Database. The sensor stayed in Room B, the bedroom next to the bathroom and the one with visible mould, for the whole capture.



| DHT22 pin | Nano 33 IoT pin |
|---|---|
| + / VCC | 3.3V |
| − / GND | GND |
| OUT / DATA | D3 |

Placement: height above the floor, distance from the window, heater and the mouldy surface.

| Component | Technology | File |
|---|---|---|
| Sensor reading | Arduino sketch, DHT22 read every 30 s, HTTPS POST to Firebase | dht22_firebase.ino |
| Data ingestion | Python script downloads readings from Firebase | live_monitor.py |
| Buffering and timestamp | Pandas DataFrame; Firebase server timestamp converted to UTC | live_monitor.py |
| Storage | Firebase Realtime Database, then local CSV | raw_data.csv |

| Component | Technology | File |
|---|---|---|
| Outdoor humidity | Bureau of Meteorology half-hourly observations, Melbourne (Olympic Park) | fetch_bom.py, bom_outdoor.csv |
| Processing | Pandas and NumPy | process.py, processed_data.csv |
| Analysis | SciPy, Matplotlib, scikit-learn | analysis.py, model.py |

Data flow: DHT22 → Nano 33 IoT → WiFi → Firebase → live_monitor.py → raw_data.csv → process.py → processed_data.csv → analysis.py.

## 1.2 Data collection execution

Collection ran from Monday 28 September 13:03 to Friday 2 October 17:48 (Melbourne time), well beyond the 30-minute minimum. The analysis window is 99.8 hours long and contains 71.7 hours of valid readings; the rest was lost in two outages. The longest uninterrupted stretch is 39.7 hours (1 October 02:07 to 2 October 17:48).

| Date | Time | Event | Notes |
|---|---|---|---|
| 28 Sep | 13:03 | First reading received in Firebase | Sensor being placed in Room B |
| 28 Sep | 13:31 | Readings continuous from here | Two short dropouts (15 and 10 min) while the sensor was positioned |
| 28 Sep | 14:00 | Analysis window starts | 30 minutes allowed for the sensor to settle |
| 29 Sep | 01:44 | Readings stop | Board offline (Uploads began failing, then the board stopped sending until it was manually reset; the sketch has no watchdog or self-reset.) |
| 29 Sep | 12:41 | Readings resume | 11-hour gap |
| 30 Sep | 09:10 | Readings stop | Board offline (Uploads began failing, then the board stopped sending until it was manually reset; the sketch has no watchdog or self-reset.) |
| 1 Oct | 01:52 | Readings resume | 17-hour gap |
| 2 Oct | 17:48 | Last reading, board unplugged | End of collection |

This log was rebuilt from the sensor timestamps after collection. No shower log was kept, so the times of individual showers are not known. Showers usually happened between 11:00 and 15:00 and between 20:00 and 01:00; this is my recollection, not a record.

## 1.3 Deviations from the Task 3 protocol

| Element | Task 3 design | What was done | Why and impact |
|---|---|---|---|
| Humidity sensors | 5 × DHT22: Room A, Room B, bathroom, hallway, outdoors | 1 × DHT22 in Room B for the whole capture | Only one sensor was available. No Room A comparison and no bathroom-to-bedroom route. |
| Shower and ventilation record | Reed switches on doors and windows | No log kept. Usual shower hours reported afterwards; Room B door/window openings detected from sudden humidity drops | I did not record shower times during collection. The per-shower measures (peak rise, damp time, decay) cannot be computed. |
| Condition assignment | Randomised per shower (NORMAL or ROUTINE) | No record of which condition each shower used | H3 cannot be tested. |
| Outdoor humidity | On-site outdoor DHT22 | Nearest BOM station, half-hourly, last 46 hours of the capture only | No second sensor. BOM keeps 72 hours online and the download was |

| Element | Task 3 design | What was done | Why and impact |
|---|---|---|---|
| | | | run once at the end. The station is several kilometres away. |
| Duration | 6 weeks, about 84 showers | 4.2 days, 71.7 hours of valid data | Time available, and two board outages (11 h and 17 h). |
| Sampling | Every 30 s | Every 30 s (median interval 31 s) | As designed. |
| Boards and link | 3 × Nano 33 IoT over WiFi | 1 × Nano 33 IoT → WiFi → Firebase → Python | Matches the Week 5 Firebase activity. |
| Analysis | Paired Wilcoxon (H1), VTT mould index (H2), mixed-effects model (H3) | Continuous-record analysis: time above critical RH, regression on outdoor humidity, Mann-Whitney U on usual shower hours vs other hours | Follows from the missing shower log and the single sensor. |
| Calibration | 24 h side by side, weeks 0 and 7 | None | One sensor, nothing to calibrate against. The DHT22's ±2–5% RH accuracy limits comparisons against fixed thresholds. |
| Consent | Signed by housemate | Not needed | I'm the sole occupant now, room B unoccupied during the capture. |

## 1.4 Data quality report

| Check | Result |
|---|---|
| Raw readings received | 8,518 (28 Sep 13:03 to 2 Oct 17:48) |
| Failed sensor reads (stored as missing) | 11 (0.13%) |
| Duplicate timestamps | 0 |
| Humidity outside 0–100% or temperature outside −10–50 °C | 0 |
| Spikes more than 8% RH from the median of 5 neighbours | 0 |
| Readings removed in the first 15 min after a long outage (re-settling) | 52 |
| Readings before the window start (sensor settling) | 61 slots |
| Gaps longer than 5 minutes | 4: 15 min and 10 min (placement), 657 min, 1,002 min |
| Analysis window on the 30 s grid | 11,978 slots = 99.8 h |
| Missing before interpolation | 3,585 slots (29.9%) |
| Interpolated (gaps of 2 minutes or less) | 214 slots |
| Still missing (long outages, left empty) | 3,371 slots (28.1%) |
| Final row count | 11,978 rows; 8,607 with a humidity value = 71.7 h |
| Readings with a matched outdoor (BOM) value | 5,517 (46.0 h) |

## 1.5 Data processing pipeline

process.py turns raw_data.csv into processed_data.csv in seven steps using Pandas and NumPy.

| Step | What the code does |
|---|---|
| Load | Reads raw_data.csv and bom_outdoor.csv into DataFrames and parses the timestamps as UTC. |
| Validate | Prints data types, missing values, failed reads, duplicate timestamps, the median sampling interval and every gap longer than 5 minutes. |
| Clean | Drops duplicates; sets out-of-range values and single-reading spikes to missing; removes the first 15 minutes after each long outage. |
| Locate | Keeps the Room B window set in config.py (28 Sep 14:00 to 2 Oct 17:49). |
| Align | Puts readings on an exact 30 s grid and interpolates gaps of up to 2 minutes; longer gaps stay missing. |
| Transform | Adds Melbourne local time, hour, time of day, a 5-minute moving average, absolute humidity ($g/m^3$), the nearest outdoor reading, a usual-shower-hours flag, and door/window events (a fall of 3% RH or more within 5 minutes). |
| Export | Writes processed_data.csv and data_quality_report.txt. |

# Part 2: Statistical analysis and visualisation

## 2.1 Descriptive statistics

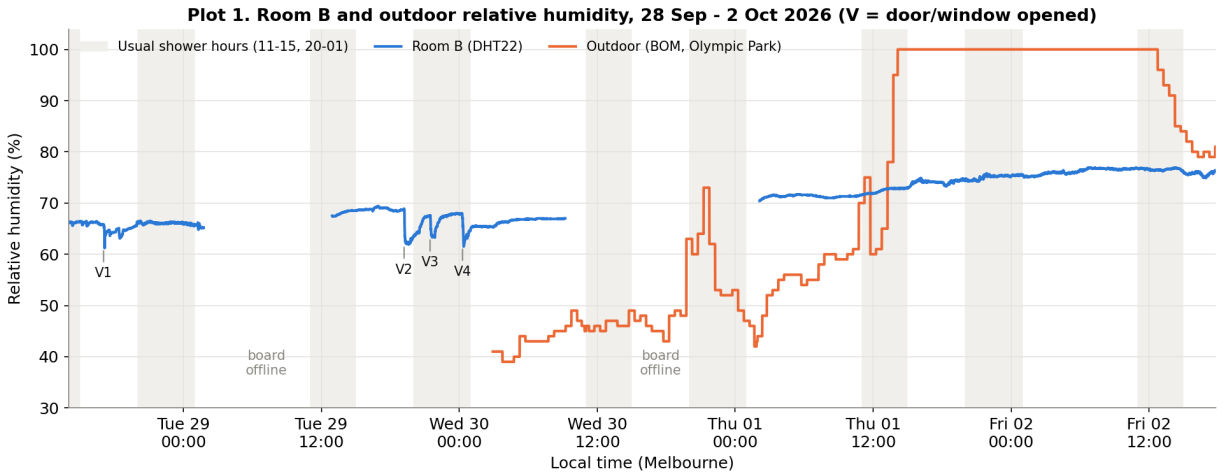
| Variable | Mean | Median | SD | Min | Max | Range | P25 | P75 | P95 | n |
|---|---|---|---|---|---|---|---|---|---|---|
| Room B relative humidity (%) | 70.8 | 71.3 | 4.3 | 61.2 | 76.9 | 15.7 | 66.4 | 75.3 | 76.7 | 8,607 |
| Room B temperature (°C) | 19.3 | 19.4 | 1.6 | 16.6 | 22.9 | 6.3 | 18.0 | 20.5 | 22.3 | 8,607 |
| Room B absolute humidity (g/m³) | 11.8 | 11.5 | 0.9 | 9.5 | 14.0 | 4.5 | 11.1 | 12.6 | 13.5 | 8,607 |
| Outdoor relative humidity (%) | 80.1 | 96.0 | 22.9 | 39.0 | 100.0 | 61.0 | 58.0 | 100.0 | 100.0 | 5,517 |
| Outdoor temperature (°C) | 16.1 | 13.9 | 3.9 | 11.9 | 23.5 | 11.6 | 12.6 | 20.1 | 21.6 | 5,517 |
| Outdoor absolute humidity (g/m³) | 10.5 | 10.8 | 1.5 | 6.7 | 14.9 | 8.2 | 10.4 | 11.1 | 12.3 | 5,517 |

Room B was damp throughout: its humidity ranged over 15.7 points (61.2–76.9%) and never fell below 60%, while outdoor humidity ranged over 61 points (39–100%). The room's temperature stayed within 16.6–22.9 °C.

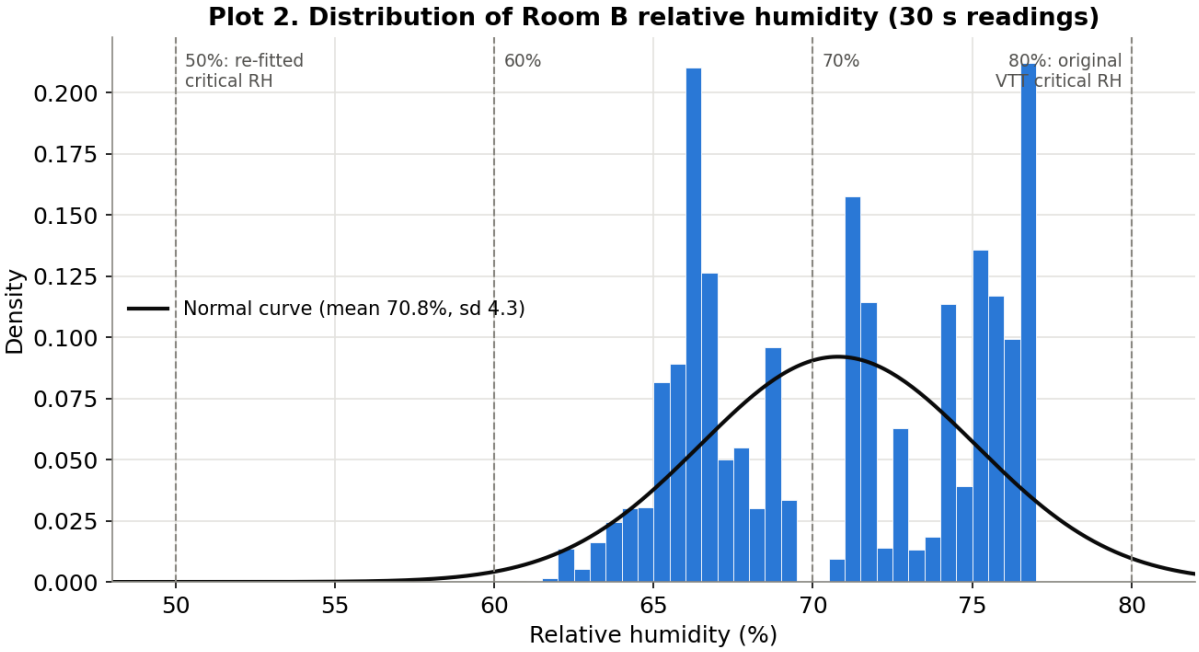
Correlation (Pearson r, hourly means, n = 47 hours with outdoor data): Room B humidity against outdoor humidity r = 0.90, against outdoor temperature r = −0.80, against Room B temperature r = −0.70. In absolute humidity the indoor–outdoor link is weaker (r = 0.41), so part of the relative-humidity link comes from both following temperature. The full matrix is below and in Plot 6.

| Pearson r | Room B RH | Room B temp | Room B abs. hum. | Outdoor RH | Outdoor temp | Outdoor abs. hum. |
|---|---|---|---|---|---|---|
| Room B RH | 1.00 | −0.70 | −0.18 | 0.90 | −0.80 | 0.71 |
| Room B temp | −0.70 | 1.00 | 0.83 | −0.68 | 0.86 | −0.10 |
| Room B abs. hum. | −0.18 | 0.83 | 1.00 | −0.24 | 0.56 | 0.41 |
| Outdoor RH | 0.90 | −0.68 | −0.24 | 1.00 | −0.92 | 0.71 |
| Outdoor temp | −0.80 | 0.86 | 0.56 | −0.92 | 1.00 | −0.38 |
| Outdoor abs. hum. | 0.71 | −0.10 | 0.41 | 0.71 | −0.38 | 1.00 |

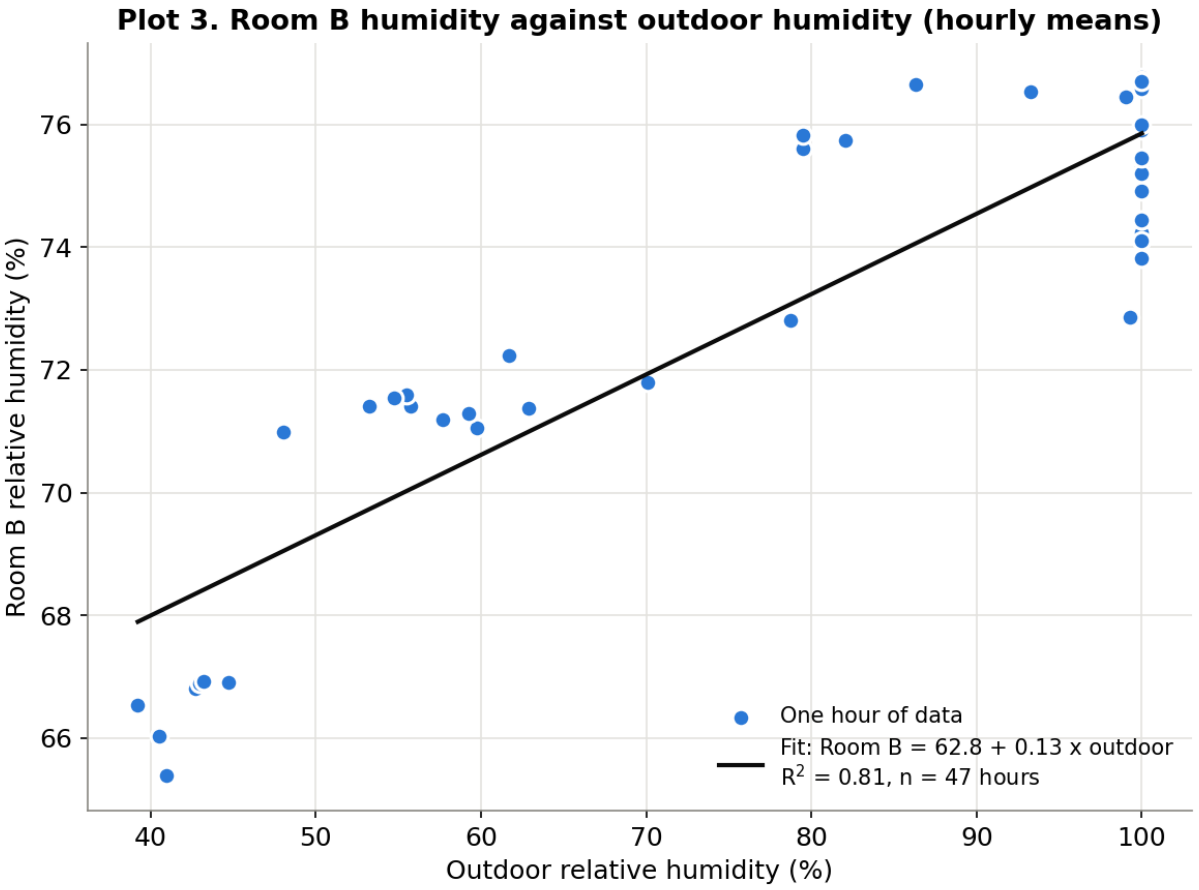
## 2.2 Visualisations



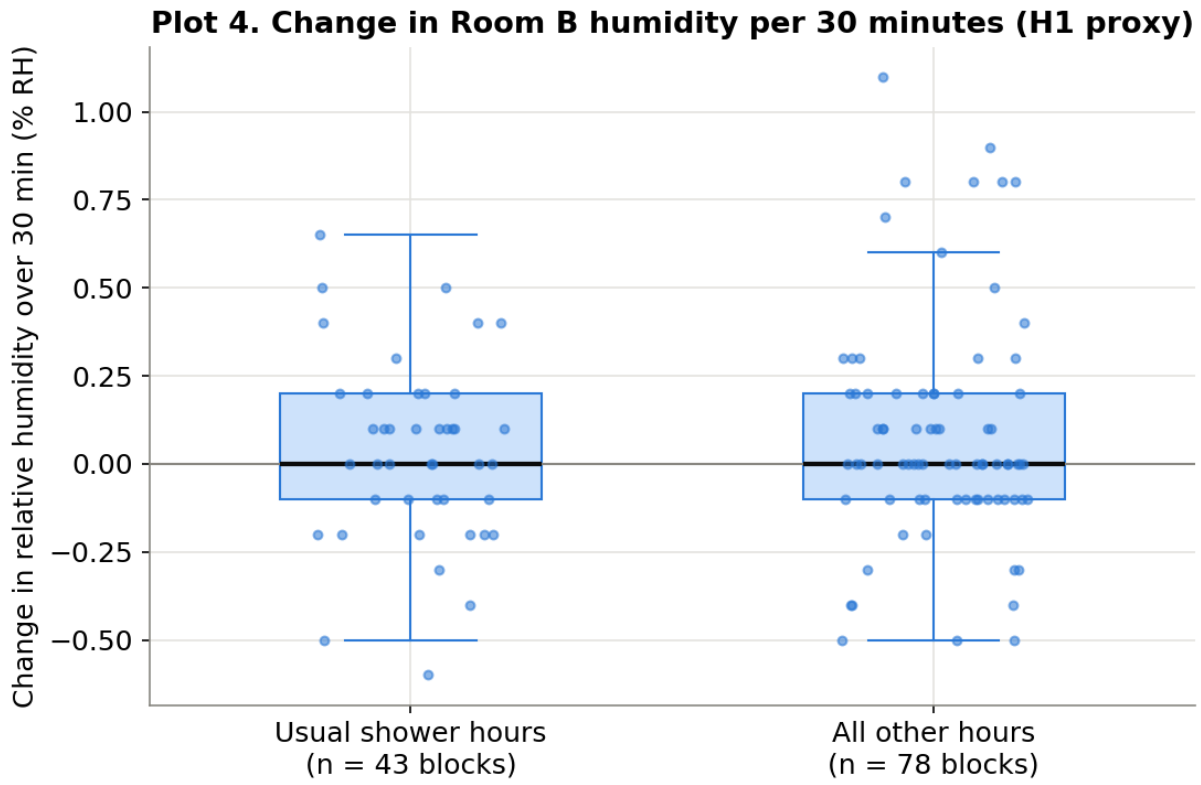
*Plot 1 (time series). Room B stays between 61% and 77% and climbs from about 66% to 76% as the weather turns wet on 1 October. No shower-sized peaks are visible; the sharp features are four drops (V1–V4) when the Room B door or window was opened.*



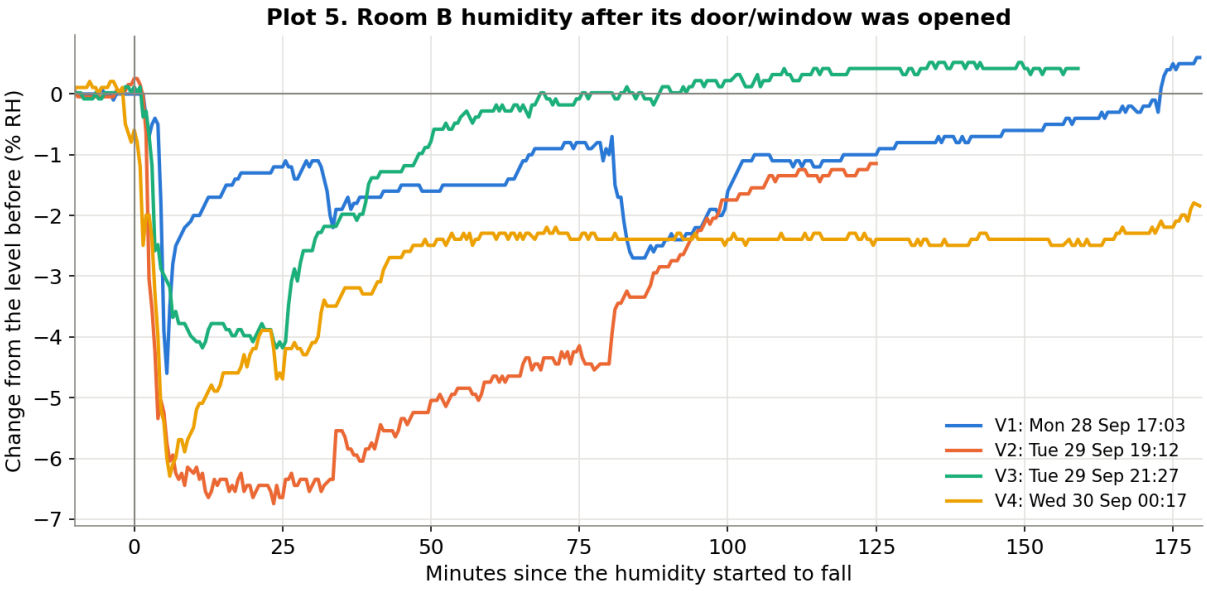

*Plot 2 (distribution). Every reading is above 60% and 55% are above 70%; none reach 80%. The two clusters are the drier first days and the wet last two days, so the normal curve fits poorly.*



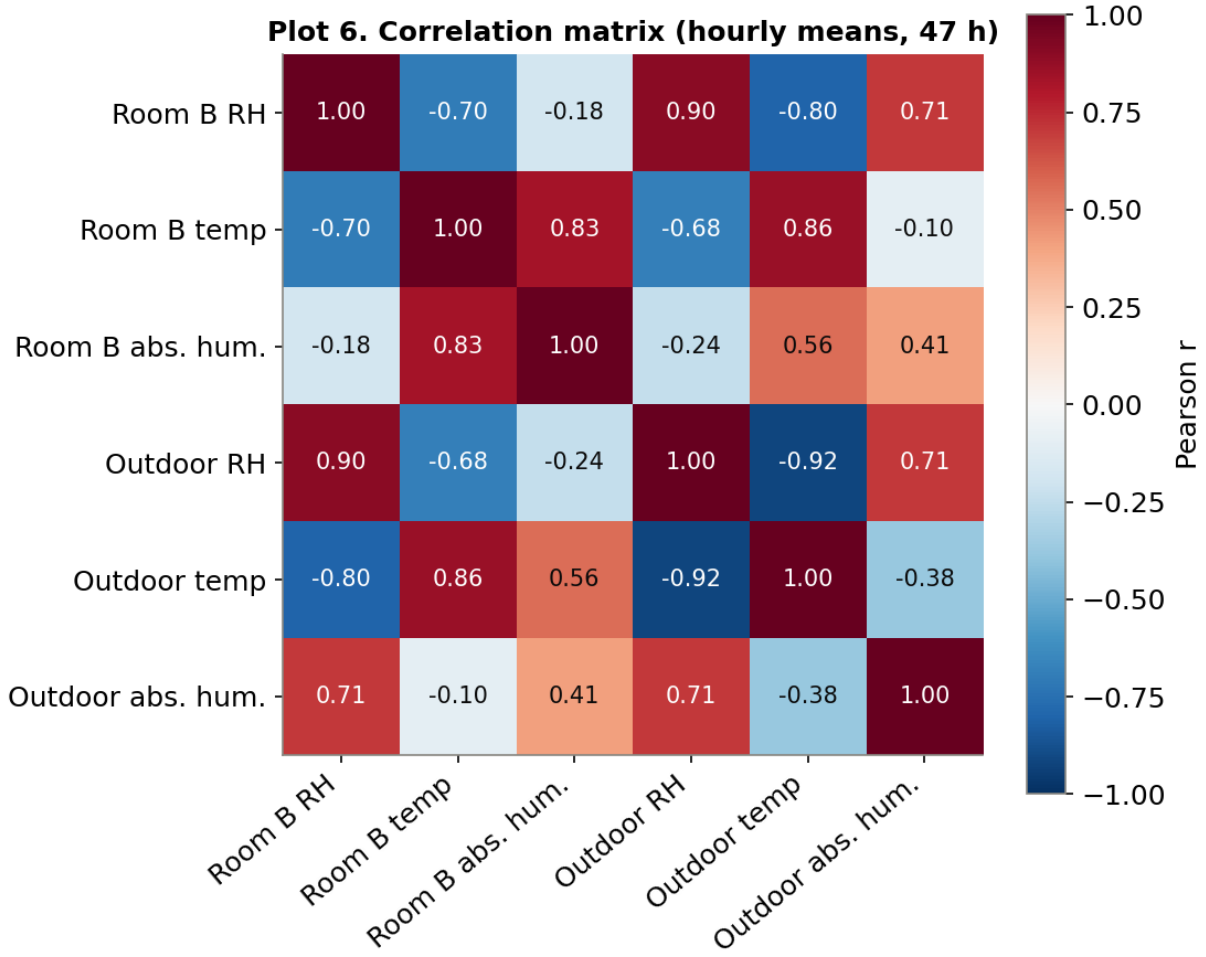

*Plot 3 (relationship). Hourly Room B humidity against outdoor humidity: $R^2 = 0.81$, about 1.3 points indoors for every 10 points outdoors.*




*Plot 4 (box plot). Change in Room B humidity per 30 minutes in the hours when showers usually happen compared with all other hours. The two groups overlap almost completely (p = 0.85).*




*Plot 5. Room B humidity after each door/window opening, relative to the level before. It falls 4–7 points within minutes and then creeps back towards the damp level.*




*Plot 6. Correlation matrix of hourly means (47 hours with outdoor data).*

## 2.3 Statistical testing and hypothesis validation

Decision rule: a hypothesis is supported when $p < 0.05$ and the direction matches; otherwise it is not supported, or not testable where the data needed was not collected.

| Hypothesis | Test | Result | Verdict |
|---|---|---|---|
| H1: after a shower Room B rises further and dries more slowly than Room A | Proxy: humidity change per 30 min, usual shower hours vs other hours, Mann-Whitney U | Median 0.0 vs 0.0 points (n = 43 vs 78 blocks), U = 1,641, p = 0.85, rank-biserial r = −0.02. Largest 15-min rise in the record: +1.0 point | Not supported by the proxy; Room A comparison not testable |
| H2: Room B's mould risk comes from staying damp, not from peaking high | Time above critical RH; regression of hourly Room B RH on outdoor RH | Above 50% RH 100% of the time, above 70% 55%, above 80% 0%. Same exposure in usual shower hours (20.6) as other hours (20.9 damp-hours per hour). $R^2 = 0.81$, $p < 0.001$, n = 47 h; on hour-to-hour changes $R^2 = 0.02$, p = 0.32 | Partly supported (Room B only) |
| H3: the routine (door closed, window open 15 min) cuts Room B damp time by a third | None possible. Described: Room B door/window openings | Median drop 5.4 points within about 9 min (n = 4); back within 1 point after 48 and 67 min in two events, still below after 2–3 h in the other two | Not testable |

Effect sizes: Room B rose about 1.3 points for every 10-point rise in outdoor humidity; opening its door or window lowered humidity by a median of 5.4 points; the difference between usual shower hours and other hours was 0.0 points.

Unexpected patterns: the largest events in the record were sudden drops, not rises, and none of them came from showers. Room B was also already above 70% from 02:07 on 1 October, about 12 hours before outdoor humidity reached 100%, so part of the step up happened during the 17-hour outage and its cause is unknown.

## 2.4 Data interpretation report

### Summary of findings

Room B was damp for the entire capture, and showers left almost no mark on it. Over 71.7 hours of valid data (28 September to 2 October, one reading every 30 seconds) its relative humidity had a median of 71.3% and a range of 61.2–76.9%. It was above 60% for 100% of the recorded time and above 70% for 55%. Outside door and window events, the largest rise in any 15 minutes was 1.0 point. The sharpest changes went the other way: four sudden drops of 4.2–6.7 points when the Room B door or window was opened. Across the capture the room climbed from about 66% to 76% as outdoor humidity went from about 42% to 96% with the rain (Plot 1).

### Hypothesis validation

H2 was partly supported: Room B's damp exposure comes from a constantly high background, not from shower peaks. The room never reached the original 80% critical humidity but was above the re-fitted 50% level 100% of the time, and its exposure was the same in the hours when showers usually happen (20.6 damp-hours per hour) as in all other hours (20.9). Hourly Room B humidity followed outdoor humidity ($R^2$ = 0.81, $p < 0.001$, n = 47 hours), rising about 1.3 points for every 10 points outdoors. Hour-to-hour changes did not follow it ($R^2$ = 0.02, p = 0.32), so this link rests on one slow weather change.

H1 was not supported by the proxy test and could not be tested as designed. Humidity change per 30 minutes was no different in usual shower hours than in other hours (median 0.0 against 0.0 points, Mann-Whitney U = 1,641, p = 0.85, rank-biserial r = −0.02, n = 43 and 78 blocks). Room A was not measured.

H3 was not testable because there is no record of which showers used the routine. Descriptively, opening the Room B door or window cut its humidity by a median of 5.4 points within about 9 minutes. Two events were back within 1 point after 48 and 67 minutes; the other two were still below their starting level 2–3 hours later.

### Limitations

The main limitation is that I did not log shower times, so the per-shower measures planned in Task 3 could not be computed. The usual shower hours are recalled afterwards and cover nine hours of each day, a weak proxy. One sensor in one room means no Room A, bathroom or hallway data. Two outages (11 and 17 hours) removed 28% of the window. Outdoor data covers only 46 hours and comes from a station several kilometres away. It rained on the last two days of the capture, and every reading above 70% comes from those days, so the time above 70% reflects one wet spell and would likely be lower in a dry week. Hourly readings are strongly autocorrelated (residual lag-1 r = 0.83), so the regression p-value is overstated. The door and window events were found with a threshold chosen after seeing the data, and their cause relies on memory. The DHT22's ±2–5% accuracy affects the threshold comparisons.

### Implications

The results suggest Room B's mould is driven by a background humidity that is high all the time and follows the weather, more than by moisture arriving from showers. The room sat 11–27 points above the 50% level at which mould risk begins in occupied homes (Menneer et al., 2022) in dry and wet weather alike. A shower routine alone is therefore unlikely to solve the problem, and briefly opening the door or window only helped for an hour or so.

### Recommendations

For the next iteration I would record every shower at the time it happens, with a push button on the board or a second DHT22 in the bathroom that timestamps showers automatically. I would add a sensor in Room A so both rooms are measured during the same shower, run the outdoor download daily, and add an alert when the board goes offline. Collection should run at least two weeks to cover several dry and wet spells. Because the background level dominates, the intervention worth testing is sustained ventilation or a dehumidifier in Room B, in addition to the shower routine.

## Part 3: Predictive model

A linear regression predicts Room B's humidity one hour ahead from its humidity now, the outdoor humidity and the outdoor temperature (model.py, 45 hourly pairs from 3 days).

| Model | $R^2$ | RMSE (% RH) |
|---|---|---|
| Linear regression, in-sample | 0.993 | 0.28 |
| Linear regression, leave-one-day-out | 0.914 | 0.94 |
| Baseline: persistence (next hour = this hour) | 0.988 | 0.35 |

Equation: RH in 1 hour = 5.97 + 0.885 × RH now + 0.022 × outdoor RH + 0.052 × outdoor temperature.

The model does not beat the baseline on unseen days (RMSE 0.94 against 0.35), so with three days of data the outdoor terms add nothing reliable. The useful result is the baseline itself: Room B's humidity changes so slowly that the best one-hour forecast is the current value, which fits the finding that the room is governed by a slow background level and not by short events.

## Part 4: Video demonstration Video online

Link to video https://deakin.au.panopto.com/Panopto/Pages/Sessions/List.aspx?folderID=b5c5db7f-1fd4-498e-b0db-b4d80066ed14

## References


[1] T. Menneer, M. Mueller, R. A. Sharpe, and S. Townley, "Modelling mould growth in domestic environments using relative humidity and temperature," *Building and Environment*, vol. 208, art. no. 108583, 2022, doi: 10.1016/j.buildenv.2021.108583.

[2] E. Vereecken and S. Roels, "Review of mould prediction models and their influence on mould risk evaluation," *Building and Environment*, vol. 51, pp. 296–310, 2012.

[3] London Office of Technology and Innovation and Greater London Authority, "Using sensors to address damp and mould in social housing," LOTI, London, U.K., 2024. [Online]. Available: https://loti.london/blog/damp-and-mould-iot-in-social-housing/

[4] A. Hukka and H. Viitanen, "A mathematical model of mould growth on wooden material," *Wood Science and Technology*, vol. 33, no. 6, pp. 475–485, 1999, doi: 10.1007/s002260050131.

[5] World Health Organization, *WHO Guidelines for Indoor Air Quality: Dampness and Mould*. Copenhagen, Denmark: WHO Regional Office for Europe, 2009.


## Acknowledgements


This assessment work was developed with assistance from the following genAI tools:

- Claude (accessed September 2026): Used to refine the code for data capture and analysis pipeline.

All genAI-generated suggestions were critically evaluated and integrated only where supported by evidence. The final design, decisions, data collection and justifications are my own.

# Appendix A: Code listings and data files

## dht22_firebase.ino (Arduino sketch)

```
// SIT225 4P - the single DHT22 sensor (Arduino Nano 33 IoT on a USB phone charger, sends over WiFi)
// Reads the DHT22 every 30 seconds and POSTs it to Firebase Realtime Database at /readings/dht22.
// Firebase stamps each reading with its own server time (".sv": "timestamp"), so the board does not
// need a clock. Which room the sensor was in is worked out later from the time windows in config.py.
//
// Wiring (DHT22 module):  + -> 3.3V,  - -> GND,  OUT -> D2
// Libraries (Tools > Manage Libraries): "DHT sensor library" (Adafruit), "WiFiNINA", "ArduinoHttpClient"
// Board: Tools > Board > Arduino SAMD Boards > Arduino NANO 33 IoT

#include <WiFiNINA.h>
#include <ArduinoHttpClient.h>
#include "DHT.h"
#include "arduino_secrets.h"

#define DHTPIN 3
#define DHTTYPE DHT22
const unsigned long SAMPLE_MS = 30000;   // 30 s, same as the 3P protocol

DHT dht(DHTPIN, DHTTYPE);
WiFiSSLClient ssl;                                // HTTPS connection
HttpClient http(ssl, SECRET_FB_HOST, 443);

unsigned long lastSample = 0;
unsigned long sent = 0, failed = 0;               // simple counters, printed for the quality report

void connectWiFi() {
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print("Connecting to WiFi ");
    Serial.println(SECRET_SSID);
    WiFi.begin(SECRET_SSID, SECRET_PASS);
    delay(8000);
  }
  Serial.print("WiFi OK, IP: ");
  Serial.println(WiFi.localIP());
}

void setup() {
  Serial.begin(9600);
  delay(1500);                    // do NOT wait for Serial: the board must run on a charger with no laptop
  pinMode(LED_BUILTIN, OUTPUT);
  dht.begin();
  connectWiFi();

  // ---- start-up self-test: tells you which part is broken ----
  Serial.print("Firebase host: '"); Serial.print(SECRET_FB_HOST); Serial.println("'");
  if (ssl.connect(SECRET_FB_HOST, 443)) {
    Serial.println("SSL test: OK (board can reach Firebase)");
  } else {
    Serial.println("SSL test: FAILED -> check host spelling, then upload the host's SSL certificate "
                   "(Tools > Upload SSL Root Certificates / Firmware Updater)");
  }
  ssl.stop();
  float h0 = dht.readHumidity();
  Serial.print("DHT22 test on pin D"); Serial.print(DHTPIN); Serial.print(": ");
  Serial.println(isnan(h0) ? String("FAILED (check wiring / pin number)") : String(h0, 1) + " % RH OK");

  lastSample = millis() - SAMPLE_MS;
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) connectWiFi();   // auto-reconnect if the router drops us

  if (millis() - lastSample >= SAMPLE_MS) {
    lastSample = millis();

    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (isnan(h) || isnan(t)) { delay(2100); h = dht.readHumidity(); t = dht.readTemperature(); }

    // A failed read is still sent (as -1 with ok:false) so the gap shows up as missing data later
    bool ok = !(isnan(h) || isnan(t));
    String body = "{\"temperature\":" + String(ok ? t : -1.0, 1) +
                  ",\"humidity\":"    + String(ok ? h : -1.0, 1) +
                  ",\"ok\":"          + String(ok ? "true" : "false") +
                  ",\"ts\":{\".sv\":\"timestamp\"}}";

    String path = "/readings/dht22.json?auth=" + String(SECRET_FB_AUTH);
    int err = http.post(path, "application/json", body);   // POST = push a new child with a unique key
    int status = (err == 0) ? http.responseStatusCode() : err;  // err -1 = could not connect to Firebase
    http.responseBody();                          // read and discard the reply so the connection is clean
    http.stop();

    if (status == 200) {
      sent++;
      digitalWrite(LED_BUILTIN, HIGH); delay(100); digitalWrite(LED_BUILTIN, LOW);  // blink = sent
    } else {
      failed++;
    }
    Serial.print(body); Serial.print("  -> HTTP "); Serial.print(status);
    Serial.print("  sent="); Serial.print(sent); Serial.print(" failed="); Serial.println(failed);
  }
}
```

## config.py

```
"""
SIT225 4P - shared settings. Edit THIS file, not the other scripts.
Every script does `from config import *`.

One DHT22 on a Nano 33 IoT. It stayed in ROOM B (the bedroom next to the bathroom) for the
whole capture, 28 Sep - 2 Oct 2026. No Room A session was run and no shower log was kept, so
the analysis treats the data as one continuous record (see the deviations table in the report).
"""
import os

# ---- Firebase (same project you used in the Week 5 activity) ------------------
FIREBASE_DATABASE_URL = os.environ.get(
    "FIREBASE_DATABASE_URL", "https://data-capture-unit-default-rtdb.firebaseio.com/")
FIREBASE_CREDENTIALS_PATH = os.environ.get(
    "FIREBASE_CREDENTIALS_PATH", "service-account.json")
FIREBASE_PATH = "/readings/dht22"                           # where the Nano 33 IoT writes

# ---- Time zone ----------------------------------------------------------------
LOCAL_TZ = "Australia/Melbourne"   # everything is stored in UTC and shown in Melbourne time

# ---- Where the sensor was, and when (local time). Fill these in as you go. ------
# Start each window ~30 min AFTER you put the sensor down, so it has settled to the room.
ROOM_B_START = "2026-09-28 14:00"   # sensor placed ~13:03-13:31, 30 min allowed to settle
ROOM_B_END = "2026-10-02 17:49"     # last reading before the board was unplugged
ROOM_A_START = None                 # no Room A session was run
ROOM_A_END = None

# ---- Files --------------------------------------------------------------------
RAW_CSV = "raw_data.csv"             # everything the sensor sent, untouched -> submit (raw)
BOM_CSV = "bom_outdoor.csv"          # outdoor humidity from BOM, appended daily
SHOWER_LOG = "shower_log.csv"        # you fill this in by hand after every shower
SCHEDULE_CSV = "shower_schedule.csv" # random condition list made BEFORE collection starts
PROCESSED_CSV = "processed_data.csv" # cleaned 30 s grid -> submit (processed)
EVENTS_CSV = "shower_events.csv"     # one row per shower (H1-H3 measures)
QUALITY_TXT = "data_quality_report.txt"
FIG_DIR = "figures"

# ---- Cleaning rules -----------------------------------------------------------
RH_RANGE = (0, 100)          # physically possible humidity, %
TEMP_RANGE = (-10, 50)       # indoor temperature, deg C (DHT22 is rated -40..80)
SPIKE_RH = 8.0               # a reading this far (% RH) from its 5-reading median is a sensor glitch
MAX_GAP_FILL_MIN = 2         # interpolate gaps up to 2 minutes; longer gaps stay missing

# ---- Shower event definitions (from the 3P revised design, Appendix B.3) ------
BASELINE_MIN = 10            # baseline = mean of the 10 minutes before the shower starts
EVENT_WINDOW_MIN = 180       # look 3 hours after the shower starts
THRESHOLD_RH = 70.0          # 3P threshold: minutes above 70 % RH
SECONDARY_THRESHOLD_RH = 60.0  # 1P showed rooms may never reach 70 %; 60 % is the fallback
DWELL_ABOVE_BASELINE = 3.0   # "still damp" = more than 3 % RH above the room's own baseline
PEAK_PHASE_EXTRA_MIN = 10    # H2: "peak phase" = shower start until 10 min after the peak

# ---- Continuous-record analysis (used because no shower log was kept) ----------
# Hours of the day when showers USUALLY happen (self-reported afterwards, not logged):
# 11:00-15:00 and 20:00-01:00. Used only as a proxy; exact shower times are unknown.
USUAL_SHOWER_HOURS = [11, 12, 13, 14, 20, 21, 22, 23, 0]
CRITICAL_RH = (50.0, 60.0, 70.0, 80.0)   # 80 = original VTT, 50 = re-fitted (Menneer et al. 2022)
VENT_DROP_RH = 3.0           # a fall of >= 3 % RH within 5 minutes = door/window opened in Room B
VENT_DROP_WINDOW_MIN = 5
VENT_RECOVERY_MAX_MIN = 180  # follow each ventilation event for up to 3 hours
```

## live_monitor.py (data ingestion)

```
"""
SIT225 4P - data ingestion from the cloud (Firebase -> Pandas -> raw_data.csv)

The Nano 33 IoT sends a reading to Firebase every 30 s. This script:
  * downloads every reading already in Firebase into a Pandas DataFrame (the buffer)
  * then, every 60 s, downloads only the NEW readings, adds a proper timestamp column,
    appends them to the buffer and re-saves raw_data.csv
  * prints the latest reading so you can watch it live (use this for the video)
  * warns if the sensor has gone quiet for more than 5 minutes (heartbeat check from 3P)

Run during collection (leave it open, Ctrl+C to stop):   python3 live_monitor.py
Run once at the end just to download everything:          python3 live_monitor.py --once
"""
import sys
import time
from datetime import datetime

import pandas as pd
import firebase_admin
from firebase_admin import credentials, db

from config import (FIREBASE_DATABASE_URL, FIREBASE_CREDENTIALS_PATH, FIREBASE_PATH,
                    RAW_CSV, LOCAL_TZ)

POLL_S = 60             # check Firebase every minute
```

```
SILENT_S = 300          # warn if the newest reading is older than 5 minutes

cred = credentials.Certificate(FIREBASE_CREDENTIALS_PATH)
firebase_admin.initialize_app(cred, {"databaseURL": FIREBASE_DATABASE_URL})
ref = db.reference(FIREBASE_PATH)



def to_frame(data):
    """Firebase dict {push_key: {temperature, humidity, ok, ts}} -> tidy DataFrame."""
    rows = []
    for key, r in (data or {}).items():
        ok = r.get("ok", True)
        rows.append({
            "firebase_key": key,
            "timestamp_utc": pd.to_datetime(r["ts"], unit="ms", utc=True),
            "temperature": r["temperature"] if ok else None,   # failed reads kept as missing
            "humidity": r["humidity"] if ok else None,
            "status": "ok" if ok else "read_error",
        })
    return pd.DataFrame(rows, columns=["firebase_key", "timestamp_utc", "temperature",
                                       "humidity", "status"])



def save(buffer):
    out = buffer.sort_values("timestamp_utc").copy()
    out["timestamp_utc"] = out["timestamp_utc"].dt.strftime("%Y-%m-%dT%H:%M:%S+00:00")
    out[["timestamp_utc", "temperature", "humidity", "status", "firebase_key"]].to_csv(RAW_CSV, index=False)



# ---- first full download ------------------------------------------------------
buffer = to_frame(ref.get())
save(buffer)
print(f"Downloaded {len(buffer)} readings -> {RAW_CSV}")
if "--once" in sys.argv or buffer.empty:
    if buffer.empty:
        print("No readings in Firebase yet - is the Nano 33 IoT on and showing HTTP 200?")
    sys.exit()

# ---- live loop ----------------------------------------------------------------
last_key = buffer["firebase_key"].max()      # push keys sort in time order
print("Live monitoring - Ctrl+C to stop")
try:
    while True:
        time.sleep(POLL_S)
        new = to_frame(ref.order_by_key().start_at(last_key).get())
        new = new[new["firebase_key"] != last_key]
        if not new.empty:
            buffer = pd.concat([buffer, new], ignore_index=True)
            last_key = new["firebase_key"].max()
            save(buffer)
        latest = buffer.sort_values("timestamp_utc").iloc[-1]
        age = (pd.Timestamp.now(tz="UTC") - latest["timestamp_utc"]).total_seconds()
        local = latest["timestamp_utc"].tz_convert(LOCAL_TZ).strftime("%a %H:%M:%S")
        print(f"{datetime.now():%H:%M:%S}  +{len(new)} new | total {len(buffer)} | latest {local}  "
              f"{latest['temperature']} C  {latest['humidity']} % RH")
        if age > SILENT_S:
            print(f"   WARNING: no reading for {int(age / 60)} min - check the charger and WiFi")
except KeyboardInterrupt:
    save(buffer)
    print(f"\nStopped. {len(buffer)} readings saved to {RAW_CSV}")
```

## fetch_bom.py (outdoor humidity)

```
"""
SIT225 4P - outdoor humidity from the Bureau of Meteorology (BOM).

3P planned an outdoor DHT22, but only two DHT22s are available, so outdoor humidity comes
from the nearest BOM station instead (a documented deviation).

BOM only keeps the LAST 72 HOURS online, so run this ONCE A DAY during collection
(and once more at the end). Each run adds new rows to bom_outdoor.csv without duplicates.

Default station: Melbourne (Olympic Park). To use a closer one, open
http://www.bom.gov.au/vic/observations/melbourne.shtml, click your station, and copy the
"IDV60901.xxxxx" code from the JSON link at the bottom of its page.

If the script is blocked (HTTP 403), open the JSON URL in your browser, File > Save As
bom_YYYYMMDD.json into this folder, and run:  python3 fetch_bom.py bom_YYYYMMDD.json
"""
import json
import os
import sys
import urllib.request

import pandas as pd
from config import BOM_CSV

STATION_URL = "http://www.bom.gov.au/fwo/IDV60901/IDV60901.95936.json"

if len(sys.argv) > 1:                        # manual fallback: a saved JSON file
    with open(sys.argv[1]) as fh:
        payload = json.load(fh)
else:
    req = urllib.request.Request(STATION_URL, headers={"User-Agent": "Mozilla/5.0"})
```

```
    with urllib.request.urlopen(req, timeout=30) as resp:
        payload = json.load(resp)

obs = pd.DataFrame(payload["observations"]["data"])
new = pd.DataFrame({
    # aifstime_utc looks like "20260928043000" and is already UTC
    "timestamp_utc": pd.to_datetime(obs["aifstime_utc"], format="%Y%m%d%H%M%S")
                       .dt.strftime("%Y-%m-%dT%H:%M:%S+00:00"),
    "outdoor_temp": obs["air_temp"],
    "outdoor_rh": obs["rel_hum"],
    "station": obs["name"],
}).dropna(subset=["outdoor_rh"])

if os.path.exists(BOM_CSV):
    new = pd.concat([pd.read_csv(BOM_CSV), new], ignore_index=True)
new = new.drop_duplicates(subset="timestamp_utc").sort_values("timestamp_utc")
new.to_csv(BOM_CSV, index=False)
print(f"{BOM_CSV} now has {len(new)} half-hourly rows "
      f"({new['timestamp_utc'].iloc[0]} to {new['timestamp_utc'].iloc[-1]} UTC)")
```

## process.py (processing pipeline)

```
"""
SIT225 4P - processing pipeline  (raw_data.csv  ->  processed_data.csv)

  1. LOAD       raw_data.csv, bom_outdoor.csv
  2. VALIDATE   missing values, duplicates, data types, sampling interval, long gaps
  3. CLEAN      remove duplicates, out-of-range values, single-reading spikes, and the
                first 15 minutes after each long outage (sensor re-settling)
  4. LOCATE     keep the Room B window from config.py (the sensor never left Room B)
  5. ALIGN      put readings on a regular 30 s grid; fill gaps <= 2 min only
  6. TRANSFORM  local time, time of day, 5-min moving average, absolute humidity,
                outdoor humidity (BOM), "usual shower hours" flag, ventilation events
  7. EXPORT     processed_data.csv + data_quality_report.txt

No shower log was kept, so readings cannot be tagged with a shower number or condition.
Instead each reading is flagged by whether it falls in the hours when showers usually
happen, and sudden humidity drops (door/window opened) are detected from the data itself.

Run:  python3 process.py
"""
import os
import numpy as np
import pandas as pd
from config import *

report = []
def say(msg=""):
    print(msg)
    report.append(str(msg))

def utc(s):
    """'2026-09-29 12:00' (Melbourne time) -> UTC timestamp."""
    return pd.Timestamp(s).tz_localize(LOCAL_TZ).tz_convert("UTC")

def abs_humidity(temp_c, rh):
    """Grams of water vapour per cubic metre of air (Magnus formula).
    Relative humidity changes when the room warms or cools; absolute humidity does not."""
    svp = 6.112 * np.exp(17.62 * temp_c / (243.12 + temp_c))      # saturation vapour pressure, hPa
    return 216.7 * (rh / 100 * svp) / (273.15 + temp_c)

# ---------------------------------------------------------------- 1. LOAD
raw = pd.read_csv(RAW_CSV)
raw["timestamp_utc"] = pd.to_datetime(raw["timestamp_utc"], utc=True, format="ISO8601")
say("=== SIT225 4P DATA QUALITY REPORT ===")
say(f"Raw file: {RAW_CSV}   shape: {raw.shape}")
say(f"First reading: {raw['timestamp_utc'].min().tz_convert(LOCAL_TZ):%a %d %b %H:%M}   "
    f"last reading: {raw['timestamp_utc'].max().tz_convert(LOCAL_TZ):%a %d %b %H:%M} ({LOCAL_TZ})")

# ---------------------------------------------------------------- 2. VALIDATE
say("\n-- Data types --")
say(raw.dtypes.to_string())
say("\n-- Missing values per column (raw) --")
say(raw.isnull().sum().to_string())
say("\n-- Read status --")
say(raw["status"].value_counts().to_string())
dups = raw.duplicated(subset=["timestamp_utc"]).sum()
say(f"\nDuplicate timestamps: {dups}")
ts = raw["timestamp_utc"].sort_values().reset_index(drop=True)
gaps = ts.diff().dt.total_seconds()
say(f"Median sampling interval {gaps.median():.0f} s; {(gaps > 300).sum()} gaps longer than 5 min")
for i in gaps[gaps > 300].index:
    say(f"   gap of {gaps[i] / 60:.0f} min: {ts[i - 1].tz_convert(LOCAL_TZ):%a %d %b %H:%M} "
        f"to {ts[i].tz_convert(LOCAL_TZ):%a %d %b %H:%M}")

# ---------------------------------------------------------------- 3. CLEAN
n0 = len(raw)
raw = raw.drop_duplicates(subset=["timestamp_utc"]).sort_values("timestamp_utc").reset_index(drop=True)
raw["temperature"] = pd.to_numeric(raw["temperature"], errors="coerce")
raw["humidity"] = pd.to_numeric(raw["humidity"], errors="coerce")
bad_rh = ~raw["humidity"].between(*RH_RANGE) & raw["humidity"].notna()
bad_t = ~raw["temperature"].between(*TEMP_RANGE) & raw["temperature"].notna()
raw.loc[bad_rh, "humidity"] = np.nan
raw.loc[bad_t, "temperature"] = np.nan
med = raw["humidity"].rolling(5, center=True, min_periods=3).median()
spike = (raw["humidity"] - med).abs() > SPIKE_RH
raw.loc[spike, "humidity"] = np.nan
```

```
# after an outage of more than 30 min the board restarts warm; drop its first 15 minutes
gap_s = raw["timestamp_utc"].diff().dt.total_seconds()
restart = raw["timestamp_utc"].where(gap_s > 1800).ffill()
settling = (raw["timestamp_utc"] - restart) < pd.Timedelta(minutes=15)
n_settle = int((settling & raw["humidity"].notna()).sum())
raw.loc[settling, ["humidity", "temperature"]] = np.nan
say("\n-- Cleaning --")
say(f"Duplicates removed: {n0 - len(raw)}")
say(f"Humidity outside {RH_RANGE} %: {int(bad_rh.sum())}   Temperature outside {TEMP_RANGE} C:
{int(bad_t.sum())}")
say(f"Humidity spikes (> {SPIKE_RH} % from rolling median of 5) removed: {int(spike.sum())}")
say(f"Readings in the first 15 min after an outage > 30 min (re-settling) removed: {n_settle}")


# ------------------------------------------------------------------ 4 + 5. LOCATE and ALIGN
grid = raw.set_index("timestamp_utc")[["temperature", "humidity"]].resample("30s").mean()
grid = grid.rename(columns={"temperature": "temp", "humidity": "rh"})
windows = [("B", ROOM_B_START, ROOM_B_END)]
if ROOM_A_START and ROOM_A_END:
    windows.append(("A", ROOM_A_START, ROOM_A_END))


parts = []
say("\n-- Location window and 30 s alignment --")
for room, start, end in windows:
    part = grid.loc[utc(start):utc(end)].copy()
    n_slots, miss0 = len(part), part["rh"].isna().sum()
    for c in ["temp", "rh"]:
        part[c] = part[c].interpolate(limit=MAX_GAP_FILL_MIN * 2, limit_area="inside")
    miss1 = part["rh"].isna().sum()
    part["room"] = room
    parts.append(part)
    say(f"Room {room}: {start} to {end}  ->  {n_slots} slots = {n_slots / 120:.1f} h | missing "
        f"{miss0} ({miss0 / max(n_slots, 1):.1%}) -> interpolated {miss0 - miss1} -> still missing "
        f"{miss1} ({miss1 / max(n_slots, 1):.1%})")
wide = pd.concat(parts)
outside = grid["rh"].notna().sum() - sum(grid.loc[utc(s):utc(e), "rh"].notna().sum() for _, s, e in windows)
say(f"Readings before the window start (sensor settling), not analysed: {outside} slots")


# ------------------------------------------------------------------ 6. TRANSFORM
wide.index.name = "timestamp_utc"
wide = wide.reset_index()
wide["timestamp_local"] = wide["timestamp_utc"].dt.tz_convert(LOCAL_TZ)
wide["hour"] = wide["timestamp_local"].dt.hour
wide["time_of_day"] = pd.cut(wide["hour"], bins=[-1, 5, 11, 17, 23],
                             labels=["night", "morning", "afternoon", "evening"])
wide["rh_5min"] = wide["rh"].rolling(10, min_periods=5).mean()
wide["abs_hum"] = abs_humidity(wide["temp"], wide["rh"])
wide["usual_shower_hours"] = wide["hour"].isin(USUAL_SHOWER_HOURS)


if os.path.exists(BOM_CSV):
    bom = pd.read_csv(BOM_CSV)
    bom["timestamp_utc"] = pd.to_datetime(bom["timestamp_utc"], utc=True, format="ISO8601")
    bom["outdoor_abs_hum"] = abs_humidity(bom["outdoor_temp"], bom["outdoor_rh"])
    wide = pd.merge_asof(wide.sort_values("timestamp_utc"),
                         bom[["timestamp_utc", "outdoor_rh", "outdoor_temp", "outdoor_abs_hum"]]
                         .sort_values("timestamp_utc"),
                         on="timestamp_utc", direction="nearest", tolerance=pd.Timedelta("35min"))
    has_out = wide["outdoor_rh"].notna() & wide["rh"].notna()
    say(f"\nOutdoor (BOM) data matched to {has_out.sum()} of {wide['rh'].notna().sum()} indoor readings "
        f"({has_out.sum() / 120:.1f} h); BOM only keeps 72 h, so earlier readings have no outdoor value")
else:
    wide["outdoor_rh"] = wide["outdoor_temp"] = wide["outdoor_abs_hum"] = np.nan
    say("WARNING: bom_outdoor.csv not found - outdoor humidity left empty")


# Ventilation events: humidity falls >= VENT_DROP_RH within VENT_DROP_WINDOW_MIN minutes
lag = VENT_DROP_WINDOW_MIN * 2                      # 30 s slots
drop = wide["rh"] - wide["rh"].shift(lag)
hit = wide.index[drop <= -VENT_DROP_RH]
wide["vent_event_id"] = pd.NA
wide["mins_since_vent"] = np.nan
starts, last = [], None
for i in hit:                                       # hits less than 30 min apart = same event
    if last is None or (wide.at[i, "timestamp_utc"] - last) > pd.Timedelta(minutes=30):
        starts.append(wide.loc[i - lag:i, "rh"].idxmax())   # the event starts where the fall began
    last = wide.at[i, "timestamp_utc"]
for n, i0 in enumerate(starts, start=1):
    t0 = wide.at[i0, "timestamp_utc"]
    mins = (wide["timestamp_utc"] - t0).dt.total_seconds() / 60
    nxt = wide.at[starts[n], "timestamp_utc"] if n < len(starts) else None
    in_win = (mins >= -BASELINE_MIN) & (mins < VENT_RECOVERY_MAX_MIN)
    if nxt is not None:
        in_win &= wide["timestamp_utc"] < nxt - pd.Timedelta(minutes=BASELINE_MIN)
    wide.loc[in_win, "vent_event_id"] = n
    wide.loc[in_win, "mins_since_vent"] = mins[in_win]
say(f"Ventilation events detected (fall >= {VENT_DROP_RH} % RH within {VENT_DROP_WINDOW_MIN} min): {len(starts)}")
for n, i0 in enumerate(starts, start=1):
    say(f"   V{n}: {wide.at[i0, 'timestamp_local']:%a %d %b %H:%M}")


# ------------------------------------------------------------------ 7. EXPORT
wide["timestamp_local"] = wide["timestamp_local"].dt.strftime("%Y-%m-%d %H:%M:%S")
wide["timestamp_utc"] = wide["timestamp_utc"].dt.strftime("%Y-%m-%dT%H:%M:%S+00:00")
cols = ["timestamp_utc", "timestamp_local", "room", "hour", "time_of_day", "usual_shower_hours",
        "temp", "rh", "rh_5min", "abs_hum", "outdoor_rh", "outdoor_temp", "outdoor_abs_hum",
        "vent_event_id", "mins_since_vent"]
wide[cols].to_csv(PROCESSED_CSV, index=False, float_format="%.3f")
say(f"\nFINAL: {len(wide)} rows in {PROCESSED_CSV}; {wide['rh'].notna().sum()} with a humidity value "
    f"= {wide['rh'].notna().sum() / 120:.1f} h of data")


with open(QUALITY_TXT, "w") as fh:
    fh.write("\n".join(report))
print(f"\nSaved {PROCESSED_CSV} and {QUALITY_TXT}")
```

## analysis.py (statistics and plots)

```
"""
SIT225 4P - statistics, hypothesis tests and plots  (processed_data.csv -> results)

The 3P design compared individual showers, but no shower log was kept during collection and
showers do not leave a clear mark in Room B's humidity. So the data is analysed as one
continuous record of Room B, and each 3P hypothesis is tested as far as that record allows:

  H2  (tested)       Room B's damp exposure comes from a constantly high background, not from
                     shower peaks -> time above critical RH, regression on outdoor humidity (R^2, p)
  H1  (proxy test)   "Room B rises after a shower" -> humidity change in the hours when showers
                     usually happen vs all other hours (Mann-Whitney U). Room A was not measured.
  H3  (not testable) No record of which showers used the routine. Described instead: what
                     happens to Room B's humidity when its door/window is opened.

Outputs
  descriptive_stats.csv, correlation_matrix.csv, vent_events.csv, results_summary.txt
  figures/fig1_timeseries.png      time series (required plot 1)
  figures/fig2_distribution.png    histogram (required plot 2)
  figures/fig3_relationship.png    Room B vs outdoor humidity (required plot 3)
  figures/fig4_shower_hours.png    box plot: humidity change, usual shower hours vs other hours
  figures/fig5_vent_events.png     humidity after each door/window opening
  figures/fig6_correlation.png     correlation heatmap

Run:  python3 analysis.py
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from scipy import stats
from config import *

os.makedirs(FIG_DIR, exist_ok=True)
out = []
def say(msg=""):
    print(msg)
    out.append(str(msg))

df = pd.read_csv(PROCESSED_CSV, parse_dates=["timestamp_local"])
df = df[df.room == "B"].copy()
ok = df.dropna(subset=["rh"])
SLOT_H = 0.5 / 60                                   # one 30 s slot, in hours

# plot style: one colour per series, grey for context
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
INK, MUTED, GRID = "#0b0b0b", "#898781", "#e4e3df"
plt.rcParams.update({"font.size": 12, "figure.dpi": 150, "axes.edgecolor": MUTED,
                     "axes.spines.top": False, "axes.spines.right": False,
                     "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.8,
                     "axes.axisbelow": True, "axes.titleweight": "bold", "axes.titlesize": 13})

# ============================================================ 2.1 DESCRIPTIVE STATISTICS
series = {"Room B relative humidity (%)": ok["rh"], "Room B temperature (C)": ok["temp"],
          "Room B absolute humidity (g/m3)": ok["abs_hum"],
          "Outdoor relative humidity (%)": ok["outdoor_rh"], "Outdoor temperature (C)": ok["outdoor_temp"],
          "Outdoor absolute humidity (g/m3)": ok["outdoor_abs_hum"]}
desc = pd.DataFrame({k: {"mean": s.mean(), "median": s.median(), "std": s.std(), "min": s.min(),
                         "max": s.max(), "range": s.max() - s.min(), "p25": s.quantile(.25),
                         "p75": s.quantile(.75), "p95": s.quantile(.95), "n": s.count()}
                     for k, s in series.items()}).T.round(2)
desc.to_csv("descriptive_stats.csv")
say("=== 2.1 DESCRIPTIVE STATISTICS (30 s readings; outdoor = BOM value matched to each reading) ===")
say(desc.to_string())
say(f"Hours of data: {len(ok) * SLOT_H:.1f} (Room B), {ok['outdoor_rh'].notna().sum() * SLOT_H:.1f} with outdoor
data")

# hourly means: used for correlation and regression so that 120 near-identical readings
# per hour are not counted as 120 independent observations
hourly = ok.set_index("timestamp_local")[["rh", "temp", "abs_hum", "outdoor_rh", "outdoor_temp",
                                          "outdoor_abs_hum", "hour"]].resample("1h").mean()
hourly = hourly[ok.set_index("timestamp_local")["rh"].resample("1h").count() >= 60]   # >= 30 min of data
both = hourly.dropna(subset=["outdoor_rh"])
corr = both.rename(columns={"rh": "Room B RH", "temp": "Room B temp", "abs_hum": "Room B abs. hum.",
                            "outdoor_rh": "Outdoor RH", "outdoor_temp": "Outdoor temp",
                            "outdoor_abs_hum": "Outdoor abs. hum."}).drop(columns="hour").corr().round(2)
corr.to_csv("correlation_matrix.csv")
say(f"\nCorrelation matrix (Pearson r, hourly means, n={len(both)} hours with outdoor data):")
say(corr.to_string())

# ============================================================ H2 (tested)
say("\n=== H2 (tested): Room B's damp exposure comes from a constantly high background humidity, "
    "not from shower peaks ===")
say("a) Time above each critical RH, and damp-hours (sum of RH minus critical RH while above it, %RH x h)")
for crit in CRITICAL_RH:
    above = ok["rh"] > crit
    dh = np.clip(ok["rh"] - crit, 0, None).sum() * SLOT_H
    say(f"   > {crit:.0f}% RH: {above.mean():6.1%} of recorded time ({above.sum() * SLOT_H:5.1f} h), damp-hours
{dh:7.1f}")
say("b) Is the exposure concentrated in the hours when showers usually happen?")
for crit in (50.0, 70.0):
    for flag, name in [(True, "usual shower hours"), (False, "all other hours")]:
        g = ok[ok["usual_shower_hours"] == flag]
        say(f"   critical {crit:.0f}%: {name:19s} {len(g) * SLOT_H:5.1f} h recorded, "
            f"{np.clip(g['rh'] - crit, 0, None).sum() * SLOT_H / (len(g) * SLOT_H):5.2f} damp-hours per hour, "
            f"mean RH {g['rh'].mean():.1f}%")
say("c) Largest short-term RISE in the whole record (a shower peak would show up here)")
rise15 = ok.set_index("timestamp_local")["rh"]
```

```
rise15 = (rise15 - rise15.shift(30, freq="30s")).dropna()          # change over 15 minutes
rebound = ok.set_index("timestamp_local")["mins_since_vent"].reindex(rise15.index).between(0, 120)
calm15 = rise15[~rebound]                                         # ignore the rebound after a door/window
opening
say(f"   biggest 15-min rise outside the 2 h after a door/window opening: +{calm15.max():.1f} % RH at "
    f"{calm15.idxmax():%a %d %b %H:%M}; 95th percentile of |15-min change|: {calm15.abs().quantile(.95):.1f} %
RH")
say(f"   (inside those 2 h the biggest rise is +{rise15[rebound].max():.1f} % RH: the room returning to its damp
level)")
say("d) Regression: hourly Room B humidity on hourly outdoor humidity")
lr = stats.linregress(both["outdoor_rh"], both["rh"])
say(f"   Room B RH = {lr.intercept:.1f} + {lr.slope:.3f} x outdoor RH;  R^2={lr.rvalue**2:.3f}, "
    f"p={lr.pvalue:.2g}, n={len(both)} hours")
resid = both["rh"] - (lr.intercept + lr.slope * both["outdoor_rh"])
say(f"   residual lag-1 autocorrelation = {resid.autocorr(1):.2f} (close to 1 = hours are not independent, "
    "so this p-value is too optimistic)")
chg = both[["rh", "outdoor_rh"]].diff()[both.index.to_series().diff() == pd.Timedelta("1h")].dropna()
lr_d = stats.linregress(chg["outdoor_rh"], chg["rh"])
say(f"   Hour-to-hour CHANGES (removes the shared slow trend): change in Room B = {lr_d.intercept:+.2f} + "
    f"{lr_d.slope:.3f} x change outdoors;  R^2={lr_d.rvalue**2:.3f}, p={lr_d.pvalue:.2g}, n={len(chg)}")
lr_ah = stats.linregress(both["outdoor_abs_hum"], both["abs_hum"])
say(f"   Same in absolute humidity (g/m3): Room B = {lr_ah.intercept:.2f} + {lr_ah.slope:.3f} x outdoor;  "
    f"R^2={lr_ah.rvalue**2:.3f}, p={lr_ah.pvalue:.2g}")
# daily averages remove most of the hour-to-hour dependence, but leave very few points
daily = ok.set_index("timestamp_local")[["rh", "outdoor_rh"]].resample("1D").mean().dropna()
say(f"   Daily means on days with outdoor data (n={len(daily)}): " +
    "; ".join(f"{d:%d %b} Room B {r.rh:.1f}% / outdoor {r.outdoor_rh:.1f}%" for d, r in daily.iterrows()))


# ============================================================ H1 (proxy test)
say("\n=== H1 (proxy test): does Room B's humidity rise in the hours when showers usually happen? ===")
say("Shower times were not logged, so 'usual shower hours' (11:00-15:00, 20:00-01:00, self-reported) "
    "stand in for them. Room A was not measured, so the A-vs-B half of H1 cannot be tested.")
blk = ok.set_index("timestamp_local")
agg = blk.resample("30min").agg(n=("rh", "count"), rh_first=("rh", "first"), rh_last=("rh", "last"),
                                ah_first=("abs_hum", "first"), ah_last=("abs_hum", "last"),
                                usual=("usual_shower_hours", "first"), vent=("mins_since_vent", "min"))
agg = agg[agg["n"] >= 54].copy()                                  # blocks with >= 90 % of readings
agg["d_rh"] = agg["rh_last"] - agg["rh_first"]
agg["d_ah"] = agg["ah_last"] - agg["ah_first"]
agg["vented"] = agg["vent"].notna() & (agg["vent"] < 120)         # within 2 h of a door/window opening
agg["usual"] = agg["usual"].astype(bool)


def mwu(label, x_shower, x_other, unit):
    u = stats.mannwhitneyu(x_shower, x_other, alternative="two-sided")
    rb = 2 * u.statistic / (len(x_shower) * len(x_other)) - 1      # +1 = shower hours always higher
    say(f"   {label}: usual shower hours median {x_shower.median():+.3f} {unit} (n={len(x_shower)} blocks), "
        f"other hours median {x_other.median():+.3f} {unit} (n={len(x_other)}), "
        f"Mann-Whitney U={u.statistic:.0f}, p={u.pvalue:.3f}, rank-biserial r={rb:+.2f}")


say("Change in humidity across each 30-minute block:")
mwu("Relative humidity, all blocks", agg.loc[agg.usual, "d_rh"], agg.loc[~agg.usual, "d_rh"], "% RH")
calm = agg[~agg.vented]
mwu("Relative humidity, excluding 2 h after door/window openings", calm.loc[calm.usual, "d_rh"],
    calm.loc[~calm.usual, "d_rh"], "% RH")
mwu("Absolute humidity, excluding 2 h after door/window openings", calm.loc[calm.usual, "d_ah"],
    calm.loc[~calm.usual, "d_ah"], "g/m3")


# ============================================================ H3 (not testable) + ventilation events
say("\n=== H3 (not testable): no record of which showers used the routine (door closed, window open) ===")
say("Described instead: Room B's humidity after each sudden drop. The occupant recalls these as times "
    "the Room B door or window was opened; they were detected from the data, not logged.")
rows = []
for vid, ev in df.dropna(subset=["vent_event_id"]).groupby("vent_event_id"):
    ev = ev.sort_values("mins_since_vent").dropna(subset=["rh"])
    pre = ev.loc[ev.mins_since_vent < 0, "rh"].mean()
    post = ev[ev.mins_since_vent >= 0]
    i_min = post["rh"].idxmin()
    after = post.loc[i_min:]
    back = after[after["rh"] >= pre - 1.0]                        # back within 1 % RH of the pre-level
    rows.append({"event": f"V{int(vid)}",
                 "start": ev.loc[ev.mins_since_vent >= 0, "timestamp_local"].iloc[0].strftime("%a %d %b %H:%M"),
                 "pre_rh": pre, "min_rh": post["rh"].min(), "drop_rh": pre - post["rh"].min(),
                 "mins_to_min": post.at[i_min, "mins_since_vent"],
                 "recovery_mins": back["mins_since_vent"].iloc[0] if len(back) else np.nan,
                 "followed_mins": post["mins_since_vent"].max(),
                 "drop_abs_hum": ev.loc[ev.mins_since_vent < 0, "abs_hum"].mean() - post["abs_hum"].min()})
vents = pd.DataFrame(rows)
vents.to_csv("vent_events.csv", index=False, float_format="%.2f")
say(vents.round(1).to_string(index=False))
say(f"Median drop {vents['drop_rh'].median():.1f} % RH, reached after a median of "
    f"{vents['mins_to_min'].median():.0f} min; back within 1 % RH of the starting level after a median of "
    f"{vents['recovery_mins'].median():.0f} min ({vents['recovery_mins'].notna().sum()} of {len(vents)} events "
    "recovered inside the time followed)")


# ============================================================ PLOTS
# Plot 1 - time series
fig, ax = plt.subplots(figsize=(13, 5.2))
t0, t1 = df["timestamp_local"].min(), df["timestamp_local"].max()
for day in pd.date_range(t0.normalize() - pd.Timedelta(days=1), t1.normalize(), freq="D"):
    for a, b in [(11, 15), (20, 25)]:                             # usual shower hours
        ax.axvspan(day + pd.Timedelta(hours=a), day + pd.Timedelta(hours=b), color="#f0efec", lw=0, zorder=0)
ax.plot(df["timestamp_local"], df["outdoor_rh"], color=ORANGE, lw=2, label="Outdoor (BOM, Olympic Park)")
ax.plot(df["timestamp_local"], df["rh"], color=BLUE, lw=2, label="Room B (DHT22)")
for _, v in vents.iterrows():
    tv = df.loc[(df.vent_event_id == int(v.event[1:])) & (df.mins_since_vent >= 0), "timestamp_local"].iloc[0]
    ax.annotate(v.event, (tv, v.min_rh), xytext=(0, -22), textcoords="offset points", ha="center",
                fontsize=10, color=INK, arrowprops={"arrowstyle": "-", "color": MUTED})
for a, b in [("2026-09-29 01:44", "2026-09-29 12:41"), ("2026-09-30 09:10", "2026-10-01 01:52")]:
    mid = pd.Timestamp(a) + (pd.Timestamp(b) - pd.Timestamp(a)) / 2
    ax.text(mid, 36, "board\noffline", ha="center", va="bottom", fontsize=10, color=MUTED)
ax.set_xlim(t0, t1); ax.set_ylim(30, 104)
ax.xaxis.set_major_locator(mdates.HourLocator(byhour=[0, 12]))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%a %d\n%H:%M"))
h, _ = ax.get_legend_handles_labels()
h.append(plt.Rectangle((0, 0), 1, 1, color="#f0efec", label="Usual shower hours (11-15, 20-01)"))
ax.legend(handles=h[::-1], loc="upper left", fontsize=10, frameon=False, ncol=3)
ax.set(title="Plot 1. Room B and outdoor relative humidity, 28 Sep - 2 Oct 2026 (V = door/window opened)",
```

```
       xlabel="Local time (Melbourne)", ylabel="Relative humidity (%)")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/fig1_timeseries.png"); plt.close(fig)


# Plot 2 - distribution
fig, ax = plt.subplots(figsize=(9, 5))
s = ok["rh"]
ax.hist(s, bins=np.arange(48, 82, 0.5), density=True, color=BLUE, edgecolor="white", linewidth=0.4)
x = np.linspace(48, 82, 300)
ax.plot(x, stats.norm.pdf(x, s.mean(), s.std()), color=INK, lw=2,
        label=f"Normal curve (mean {s.mean():.1f}%, sd {s.std():.1f})")
for crit, lab in [(50, "50%: re-fitted\ncritical RH"), (60, "60%"), (70, "70%"), (80, "80%: original\nVTT critical
RH")]:
    ax.axvline(crit, color=MUTED, lw=1, ls="--")
    ax.text(crit + (-0.3 if crit == 80 else 0.3), ax.get_ylim()[1] * 0.97, lab, fontsize=9, color="#52514e",
            va="top", ha="right" if crit == 80 else "left")
ax.set(title="Plot 2. Distribution of Room B relative humidity (30 s readings)",
       xlabel="Relative humidity (%)", ylabel="Density", xlim=(48, 82))
ax.legend(loc="center left", fontsize=10, frameon=False)
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/fig2_distribution.png"); plt.close(fig)


# Plot 3 - relationship: Room B vs outdoor humidity (hourly means)
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(both["outdoor_rh"], both["rh"], s=64, color=BLUE, edgecolor="white", linewidth=1.5, label="One hour of
data")
xs = np.linspace(both["outdoor_rh"].min(), both["outdoor_rh"].max(), 50)
ax.plot(xs, lr.intercept + lr.slope * xs, color=INK, lw=2,
        label=f"Fit: Room B = {lr.intercept:.1f} + {lr.slope:.2f} x outdoor\nR$^2$ = {lr.rvalue**2:.2f}, n =
{len(both)} hours")
ax.set(title="Plot 3. Room B humidity against outdoor humidity (hourly means)",
       xlabel="Outdoor relative humidity (%)", ylabel="Room B relative humidity (%)")
ax.legend(loc="lower right", fontsize=10, frameon=False)
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/fig3_relationship.png"); plt.close(fig)


# Plot 4 - box plot: humidity change per 30-min block, usual shower hours vs other hours
fig, ax = plt.subplots(figsize=(7.5, 5))
data = [calm.loc[calm.usual, "d_rh"], calm.loc[~calm.usual, "d_rh"]]
bp = ax.boxplot(data, widths=0.5, patch_artist=True, showfliers=False,
                medianprops={"color": INK, "lw": 2}, boxprops={"facecolor": "#cde2fb", "edgecolor": BLUE},
                whiskerprops={"color": BLUE}, capprops={"color": BLUE})
rng = np.random.default_rng(0)
for i, d in enumerate(data, start=1):
    ax.scatter(np.full(len(d), i) + rng.uniform(-.18, .18, len(d)), d, s=14, color=BLUE, alpha=.55, zorder=3)
ax.axhline(0, color=MUTED, lw=1)
ax.set_xticks([1, 2], [f"Usual shower hours\n(n = {len(data[0])} blocks)", f"All other hours\n(n = {len(data[1])}
blocks)"])
ax.set(title="Plot 4. Change in Room B humidity per 30 minutes (H1 proxy)",
       ylabel="Change in relative humidity over 30 min (% RH)")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/fig4_shower_hours.png"); plt.close(fig)


# Plot 5 - humidity after each door/window opening
fig, ax = plt.subplots(figsize=(10, 5))
ev_all = df.dropna(subset=["vent_event_id", "rh"])
for (vid, ev), colour in zip(ev_all.groupby("vent_event_id"), [BLUE, ORANGE, AQUA, YELLOW]):
    pre = ev.loc[ev.mins_since_vent < 0, "rh"].mean()
    v = vents[vents.event == f"V{int(vid)}"].iloc[0]
    ax.plot(ev["mins_since_vent"], ev["rh"] - pre, color=colour, lw=2, label=f"{v.event}: {v.start}")
ax.axhline(0, color=MUTED, lw=1); ax.axvline(0, color=MUTED, lw=1)
ax.set(title="Plot 5. Room B humidity after its door/window was opened",
       xlabel="Minutes since the humidity started to fall", ylabel="Change from the level before (% RH)",
       xlim=(-10, 180))
ax.legend(loc="lower right", fontsize=10, frameon=False)
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/fig5_vent_events.png"); plt.close(fig)


# Plot 6 - correlation heatmap
fig, ax = plt.subplots(figsize=(7.5, 6.3))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(corr)), corr.columns, rotation=40, ha="right")
ax.set_yticks(range(len(corr)), corr.index)
ax.grid(False)
for i in range(len(corr)):
    for j in range(len(corr)):
        val = corr.iloc[i, j]
        ax.text(j, i, f"{val:.2f}", ha="center", va="center", fontsize=10,
                color="white" if abs(val) > 0.6 else INK)
fig.colorbar(im, label="Pearson r")
ax.set_title(f"Plot 6. Correlation matrix (hourly means, {len(both)} h)", fontsize=12)
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/fig6_correlation.png"); plt.close(fig)

with open("results_summary.txt", "w") as fh:
    fh.write("\n".join(out))
print(f"\nSaved results_summary.txt, descriptive_stats.csv, correlation_matrix.csv, vent_events.csv "
      f"and 6 plots in {FIG_DIR}/")
```

## model.py (predictive model)

```
"""
SIT225 4P - Part 3 (OPTIONAL): predictive model.

Question: can Room B's humidity one hour from now be predicted from its humidity now and the
outdoor weather now?

Linear regression (scikit-learn) on hourly means. It is compared with the simplest possible
forecast, "persistence" (next hour = this hour), because a room's humidity changes slowly and
any useful model has to beat that. Tested with leave-one-day-out cross-validation: trained on
the other days, tested on a day it never saw.

Run:  python3 model.py
"""
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.model_selection import LeaveOneGroupOut, cross_val_predict
from config import PROCESSED_CSV


df = pd.read_csv(PROCESSED_CSV, parse_dates=["timestamp_local"]).dropna(subset=["rh"])
cnt = df.set_index("timestamp_local")["rh"].resample("1h").count()
h = df.set_index("timestamp_local")[["rh", "outdoor_rh", "outdoor_temp"]].resample("1h").mean()
h = h[cnt >= 60]                                          # hours with at least 30 min of data
h["rh_next"] = h["rh"].shift(-1, freq="1h").reindex(h.index)   # the value exactly one hour later
h = h.dropna()


features = ["rh", "outdoor_rh", "outdoor_temp"]
X, y = h[features].values, h["rh_next"].values
days = h.index.strftime("%Y-%m-%d").values


rmse = lambda a, b: np.sqrt(mean_squared_error(a, b))
model = LinearRegression().fit(X, y)
pred_in = model.predict(X)
pred_cv = cross_val_predict(LinearRegression(), X, y, groups=days, cv=LeaveOneGroupOut())
persist = h["rh"].values                                  # baseline: next hour = this hour


print(f"Rows used: {len(h)} hourly pairs from {len(set(days))} days ({', '.join(sorted(set(days)))})")
print(f"Linear regression, in-sample:          R^2 = {r2_score(y, pred_in):.3f}, RMSE = {rmse(y, pred_in):.2f} %
RH")
print(f"Linear regression, leave-one-day-out:  R^2 = {r2_score(y, pred_cv):.3f}, RMSE = {rmse(y, pred_cv):.2f} %
RH")
print(f"Baseline to beat (persistence):        R^2 = {r2_score(y, persist):.3f}, RMSE = {rmse(y, persist):.2f} %
RH")
print("\nEquation: RH in 1 hour = {:.2f}".format(model.intercept_) +
      "".join(f" {c:+.3f} x {f}" for c, f in zip(model.coef_, features)))
```

## raw_data.csv (raw data)

raw_data.csv is every reading the DHT22 sent to Firebase, saved untouched by live_monitor.py. It has 8,518 rows and 5 columns, from 2026-09-28 03:03 to 2026-10-02 07:48 UTC. The first 15 rows are shown below.

| timestamp_utc | temperature | humidity | status | firebase_key |
|---|---|---|---|---|
| 2026-09-28T03:03:50+00:00 | 22.7 | 62.3 | ok | -P2_y-53giXO91kBYm3t |
| 2026-09-28T03:04:38+00:00 | 22.7 | 59.7 | ok | -P2_yAlwnMvk1eBLNDtC |
| 2026-09-28T03:19:17+00:00 | | | read_error | -P2a0XJjASt4McKVzB2u |
| 2026-09-28T03:19:50+00:00 | | | read_error | -P2a0eORgFRrjCcdftgA |
| 2026-09-28T03:20:22+00:00 | | | read_error | -P2a0m6_HowRl7lPusQU |
| 2026-09-28T03:20:50+00:00 | 21.3 | 66.7 | ok | -P2a0staAee7BTezpn3k |
| 2026-09-28T03:21:19+00:00 | 21.3 | 66.9 | ok | -P2a1-BidujViJql912A |
| 2026-09-28T03:31:00+00:00 | 21.3 | 63.8 | ok | -P2a3Cs0QiPN2XiOB_KV |
| 2026-09-28T03:31:39+00:00 | 21.2 | 64.4 | ok | -P2a3MLtw2RoOwwqgvKc |
| 2026-09-28T03:32:09+00:00 | 21.3 | 66.8 | ok | -P2a3TootXDDlteF-ZW_ |
| 2026-09-28T03:32:40+00:00 | 21.4 | 64.2 | ok | -P2a3aIHx84OyrhHxsHm |
| 2026-09-28T03:33:10+00:00 | 21.4 | 64.6 | ok | -P2a3hnI7ZH4c56kHgaz |
| 2026-09-28T03:33:42+00:00 | 21.4 | 64.3 | ok | -P2a3pNpdDkrCZmkTj7L |
| 2026-09-28T03:34:12+00:00 | 21.4 | 63.9 | ok | -P2a3wljyejERelP4AOf |
| 2026-09-28T03:34:43+00:00 | 21.4 | 64.6 | ok | -P2a43Gh9x_rKsGvxyvL |

## processed_data.csv (processed data)

processed_data.csv is the output of process.py: the cleaned readings on a 30-second grid with the derived columns used in the analysis. It has 11,978 rows and 15 columns, from 2026-09-28 14:00 to 2026-10-02 17:48 local time. The first 15 rows are shown below; empty cells are values that were missing or not yet defined at that point (the 5-minute rolling average needs earlier readings, and the outdoor readings are hourly).

```
timestamp_utc,timestamp_local,room,hour,time_of_day,usual_shower_hours,temp,rh,rh_5min,abs_hum,outdoor_rh,outdoor_temp,outdoor_abs_hum,vent_event_id,mins_since_vent
2026-09-28T04:00:00+00:00,2026-09-28 14:00:00,B,14,afternoon,True,20.600,66.200,,11.821,,,,,
2026-09-28T04:00:30+00:00,2026-09-28 14:00:30,B,14,afternoon,True,20.600,66.300,,11.839,,,,,
2026-09-28T04:01:00+00:00,2026-09-28 14:01:00,B,14,afternoon,True,20.600,66.200,,11.821,,,,,
2026-09-28T04:01:30+00:00,2026-09-28 14:01:30,B,14,afternoon,True,20.600,66.200,,11.821,,,,,
2026-09-28T04:02:00+00:00,2026-09-28 14:02:00,B,14,afternoon,True,20.600,66.200,66.220,11.821,,,,
2026-09-28T04:02:30+00:00,2026-09-28 14:02:30,B,14,afternoon,True,20.600,66.200,66.217,11.821,,,,
2026-09-28T04:03:00+00:00,2026-09-28 14:03:00,B,14,afternoon,True,20.600,66.100,66.200,11.803,,,,
2026-09-28T04:03:30+00:00,2026-09-28 14:03:30,B,14,afternoon,True,20.600,66.200,66.200,11.821,,,,
2026-09-28T04:04:00+00:00,2026-09-28 14:04:00,B,14,afternoon,True,20.600,66.200,66.200,11.821,,,,
2026-09-28T04:04:30+00:00,2026-09-28 14:04:30,B,14,afternoon,True,20.600,66.200,66.200,11.821,,,,
2026-09-28T04:05:00+00:00,2026-09-28 14:05:00,B,14,afternoon,True,20.600,66.300,66.210,11.839,,,,
2026-09-28T04:05:30+00:00,2026-09-28 14:05:30,B,14,afternoon,True,20.600,66.200,66.200,11.821,,,,
2026-09-28T04:06:00+00:00,2026-09-28 14:06:00,B,14,afternoon,True,20.600,66.200,66.200,11.821,,,,
2026-09-28T04:06:30+00:00,2026-09-28 14:06:30,B,14,afternoon,True,20.600,66.300,66.210,11.839,,,,
2026-09-28T04:07:00+00:00,2026-09-28 14:07:00,B,14,afternoon,True,20.600,66.200,66.210,11.821,,,,
```

**Appendix B: Week 8 activity sheets**

Student name: Chakriya Thida An

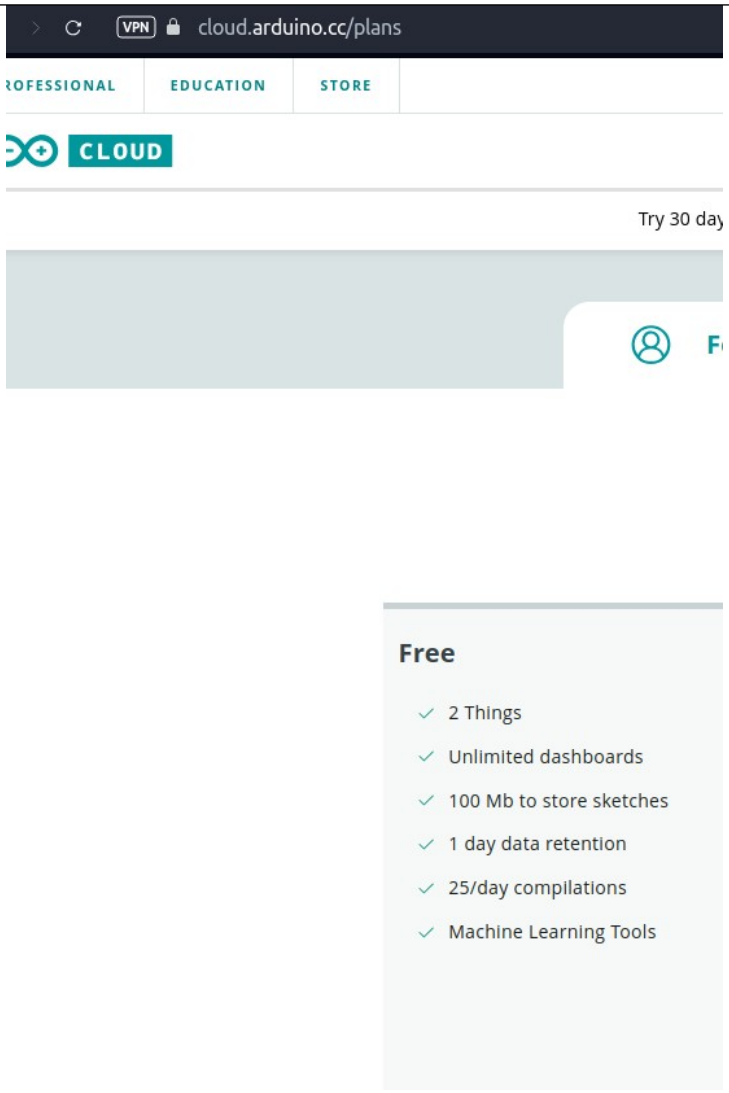
Student ID: 224753555

# SIT225: Data Capture Technologies

## Activity 8.1: Using smartphone to capture sensor data

### Steps

| Step | Action |
|---|---|
| 1 | **Install App**:<br>To use the Arduino IoT Remote app, visit Google Play / App Store and search for "Arduino IoT Remote".<br><br>After installing the app, you will need to log in to your Arduino account.<br><br>After you login, you will discover all your dashboards (if you have any), in the main menu. Based on the app version, home screen may vary. There will be 3 tabs at the bottom – Dashboards, Devices and Activity. You can follow the tutorial (https://docs.arduino.cc/arduino-cloud/iot-remote-app/getting-started ). |
| 2 | **Add device**:<br>Tap into the Devices tab. You will be able to create a new device. Alternatively, you can your profile (top right corner), in the settings section, you will see “Phone as device” which you can turn ON if it is OFF. There, you can select sensors in your smartphone such as accelerometer linear, accelerometer x/y/z and GPS among others.<br><br>Note: A free account is enough for this experiment. If you are asked to upgrade your account, you can remove all other Things from your Arduino IoT Cloud account since the Free account allows at most 2 Things to configure, see below image. |




The add device wizard will allow you to setup your sensor and also create a dashboard which you can see in your smartphone app. If you login to Arduino IoT Cloud in web browser, you can see the dashboard for your smartphone is already created.

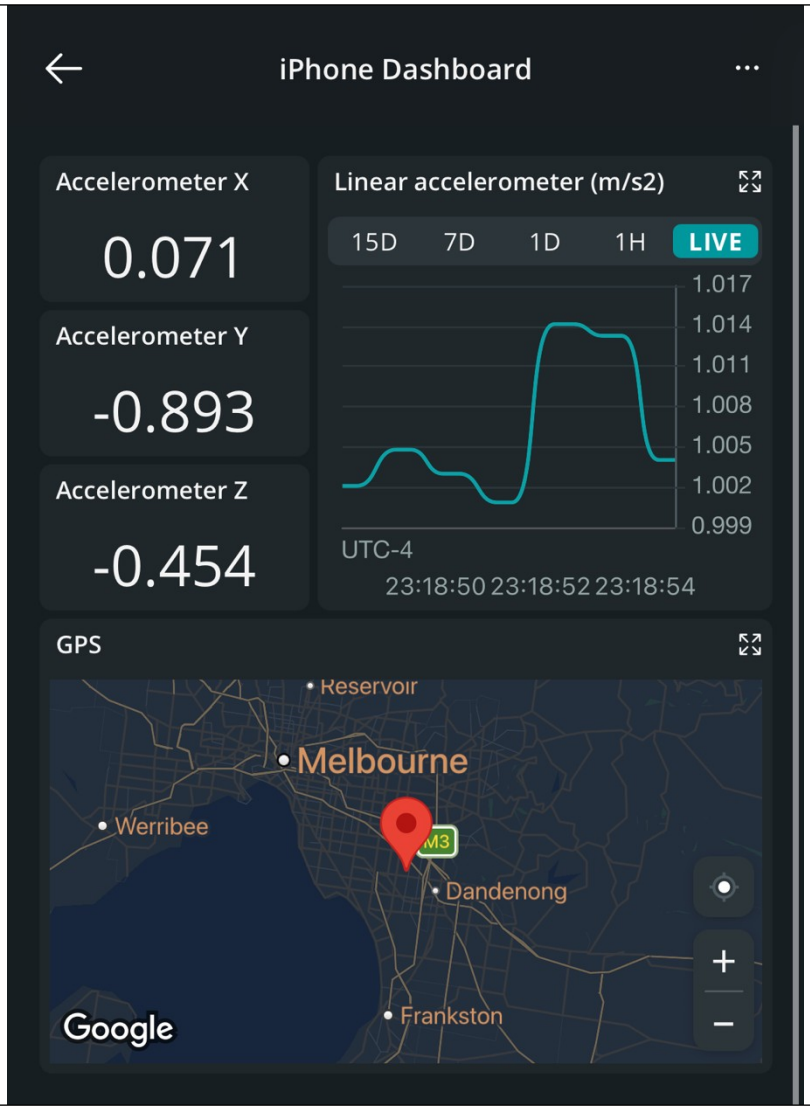
3

**Keep your smartphone screen ON for a while**:
Keep data coming through your smartphone for 10-15 minutes. During this time, keep moving your smartphone in a pattern so the accelerometer data can be analysed to discover the pattern.

You can download data from the Ardiuno Cloud dashboard page by clicking on a download icon at top right corner which shows – Download historic data. A data download link will be sent to your account email from there you can download data.

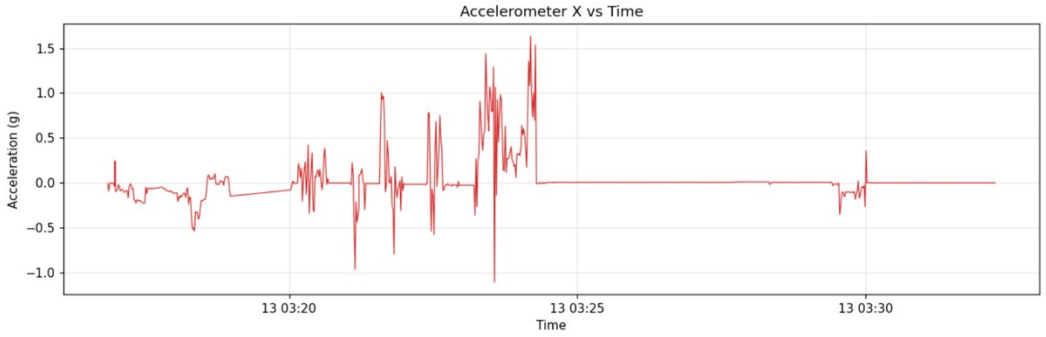
Question: Take a screenshot of your Ardiuno Cloud Dashboard where smartphone data is streaming and paste it here.

Answer: The screenshot below shows my iPhone Dashboard in the Arduino IoT Remote app with the smartphone data streaming: the live Accelerometer X, Y and Z values, the linear accelerometer chart in LIVE mode and the GPS location.

| | |
|---|---|
| |  |
| 4 | **Plot accelerometer data**:<br>The zipped data file you downloaded from the cloud contains separate files per variable including accelerometer_linear, accelerometer_x, accelerometer_y, accelerometer_z and Gps. Each file has 2 columns – time and value.<br><br>Question: Open Jupyter Notebook by using command line, go to the data folder and write command ($ jupyter lab). Using Pandas, read CSV file and fetch the data column for accelerometer_x and plot it using Python plotting library (matplotlib or any other convenient for you). Repeat the plotting process for accelerometer y and z to have 3 separate graphs. Now create a fourth graph with all 3 variables x, y and z. Screenshot the 4 graphs and paste here.<br><br>Answer: |

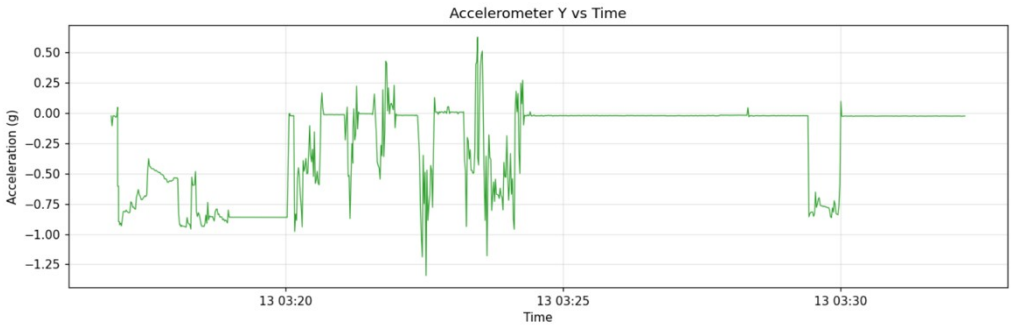
## 3. Graph 1 — Accelerometer X

1]:
```
plt.plot(acc_x["time"], acc_x["value"], color="tab:red", linewidth=0.8)
plt.title("Accelerometer X vs Time")
plt.xlabel("Time")
plt.ylabel("Acceleration (g)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



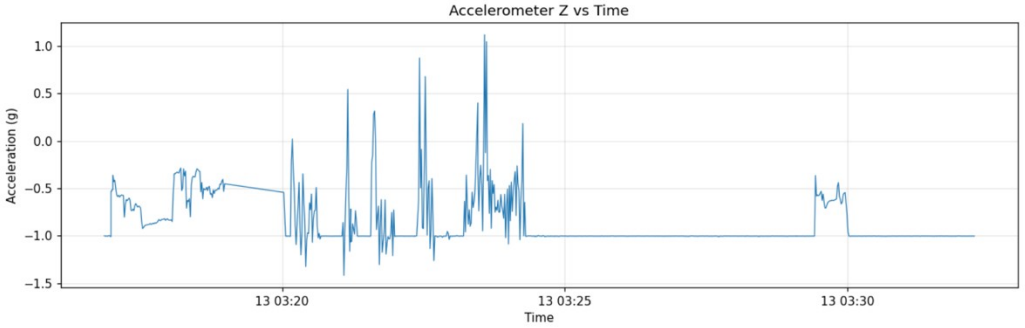

## 4. Graph 2 — Accelerometer Y

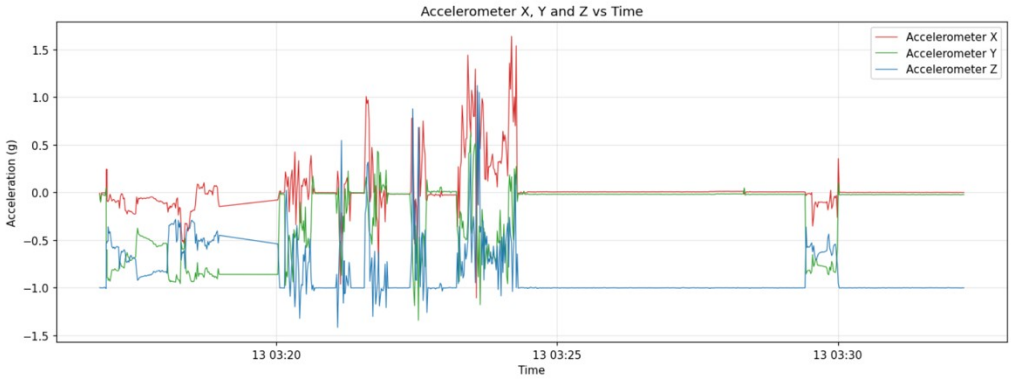
[13]:
```
plt.plot(acc_y["time"], acc_y["value"], color="tab:green", linewidth=0.8)
plt.title("Accelerometer Y vs Time")
plt.xlabel("Time")
plt.ylabel("Acceleration (g)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```




## 5. Graph 3 — Accelerometer Z

15]:
```
plt.plot(acc_z["time"], acc_z["value"], color="tab:blue", linewidth=0.8)
plt.title("Accelerometer Z vs Time")
plt.xlabel("Time")
plt.ylabel("Acceleration (g)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



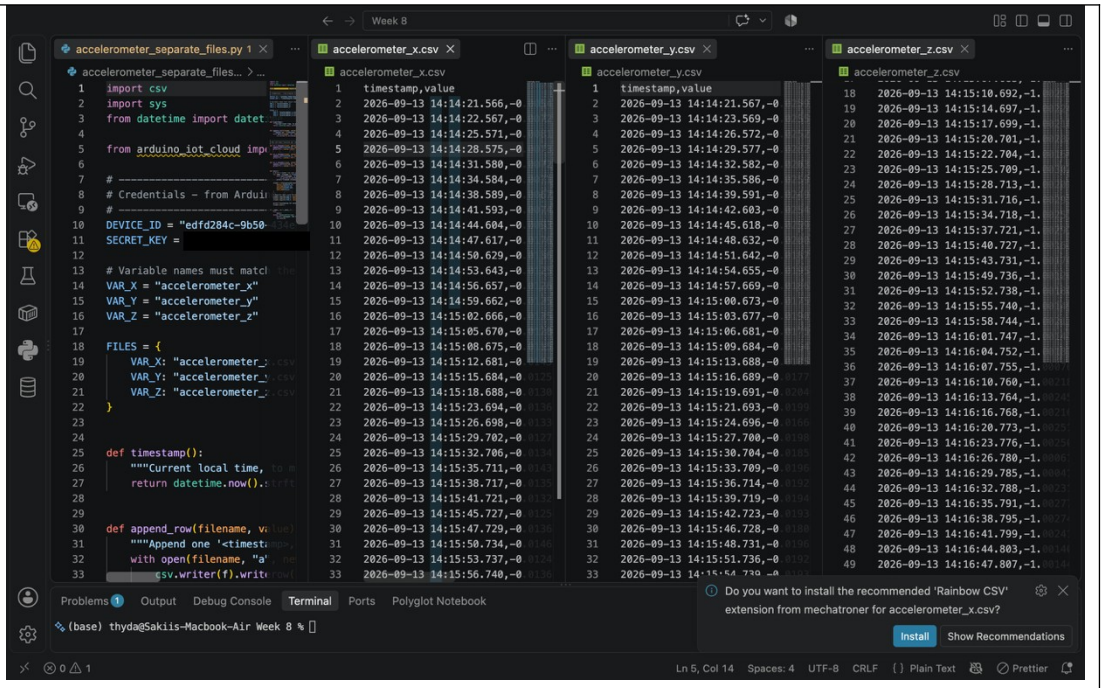
| | |
|---|---|
| | [17]:<br>```<br>plt.figure(figsize=(13, 5))<br>plt.plot(acc_x["time"], acc_x["value"], label="Accelerometer X", linewidth=0.8, color="tab:red")<br>plt.plot(acc_y["time"], acc_y["value"], label="Accelerometer Y", linewidth=0.8, color="tab:green")<br>plt.plot(acc_z["time"], acc_z["value"], label="Accelerometer Z", linewidth=0.8, color="tab:blue")<br>plt.title("Accelerometer X, Y and Z vs Time")<br>plt.xlabel("Time")<br>plt.ylabel("Acceleration (g)")<br>plt.legend(loc="upper right")<br>plt.grid(alpha=0.3)<br>plt.tight_layout()<br>plt.show()<br>```<br>  |
| 5 | Question: Analyse accelerometer variables to find any repeating pattern. Remember that you were repeatedly moving your phone in a single pattern which should be manifested in the graphs. Justify your answer.<br><br>Answer: The three accelerometer graphs show a clear repeating pattern. Over the 4-minute recording, there are six bursts of activity, with each burst lasting around 16–35 seconds. These bursts are separated by periods where the phone is almost completely still. During these periods, Z stays at around −1.0 g, while X and Y are close to 0 g. This suggests that the phone was lying flat and only measuring the effect of gravity. Each burst appears to represent one repetition of the movement.<br><br>During each burst, all three axes change together. The X-axis has the largest range, reaching up to around 2.5 g, while the Z-axis changes between approximately −1.4 g and +1.1 g. By counting the zero crossings of the mean-removed signal, the movement appears to have a cycle of around 5–6 seconds on the X and Y axes. This suggests that the movement was repeated at a similar rate within each burst. The fact that Z consistently returns to −1.0 g between the bursts also suggests that the phone was placed back in the same resting position each time.<br><br>The autocorrelation of the individual axes within each burst only shows weak peaks (r ≈ 0.2–0.3), instead of a strong periodic peak. This is likely because of the sampling rate of the data rather than the movement itself. The logged sampling rate was only around 2 Hz, which means that, based on the Nyquist criterion, signals faster than 1 Hz cannot be properly captured. These faster parts of the movement may therefore appear as noise. Overall, the repetition between the bursts is clear, but the detailed movement within each burst was not captured very well. A higher publish rate for the Arduino Cloud variable would help capture the movement in more detail. |

# Activity 8.2: Receive smartphone sensor data from Python script

You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that speaks Python. In activity 3.2, you have configured custom Python board and created a cloud variable that was synced to your Arduino Thing such as DHT22 sensor variables.
In this activity, you will need to synchronise smartphone's accelerometer x, y and z variables to Python script. If you can recall, you have already done a similar function in Activity 3.2.

### Steps:

| Step | Action |
|---|---|
| 1 | Configure Python board in Arduino Cloud and create a Thing where define 3 variables at a time and sync to corresponding accelerometer variable of smartphone Thing. |
| 2 | Write Python script to keep listening to data from the 3 variables to come through. You may need to create 3 call-back functions – a single function per variable (x, y and z). |
| 3 | Question: Keep storing each variable data in a separate file. Append each value with a timestamp so each data reading forms a comma separated line - <timestamp>, <data-value>. New data is written in a separate line. Keep storing them in a CSV file, where there will be 3 separate files. Screenshot your Python script here and screenshot the files opened side-by-side you have created and paste it here.<br><br>Answer: accelerometer_separate_files.py registers one call-back function per variable (on_x_changed, on_y_changed and on_z_changed), and each call-back appends a <timestamp>,<value> line to its own CSV file. This produced accelerometer_x.csv (166 readings), accelerometer_y.csv (167 readings) and accelerometer_z.csv (166 readings), recorded on 13 September 2026 from 14:14:21 to 14:23:11. The screenshot shows the script next to the three files opened side by side. The full script and the first rows of each file are listed below it. |




## Python script: accelerometer_separate_files.py

```
import csv
import sys
from datetime import datetime

from arduino_iot_cloud import ArduinoCloudClient

# ----------------------------------------------------------------------------
# Credentials - from Arduino Cloud > Devices > your Python device
# ----------------------------------------------------------------------------
DEVICE_ID = "edfd284c-9b50-434e-8552-528b0c46fc73"
SECRET_KEY = "<hidden for submission>"

# Variable names must match the variable names in your Python Thing exactly
VAR_X = "accelerometer_x"
VAR_Y = "accelerometer_y"
VAR_Z = "accelerometer_z"

FILES = {
    VAR_X: "accelerometer_x.csv",
    VAR_Y: "accelerometer_y.csv",
    VAR_Z: "accelerometer_z.csv",
}


def timestamp():
    """Current local time, to millisecond precision."""
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]


def append_row(filename, value):
    """Append one '<timestamp>,<value>' line to the given CSV file."""
    with open(filename, "a", newline="") as f:
        csv.writer(f).writerow([timestamp(), value])


# ----------------------------------------------------------------------------
# One call-back function per variable
# ----------------------------------------------------------------------------
def on_x_changed(client, value):
    append_row(FILES[VAR_X], value)
    print(f"x = {value:+.4f}")


def on_y_changed(client, value):
    append_row(FILES[VAR_Y], value)
    print(f"y = {value:+.4f}")

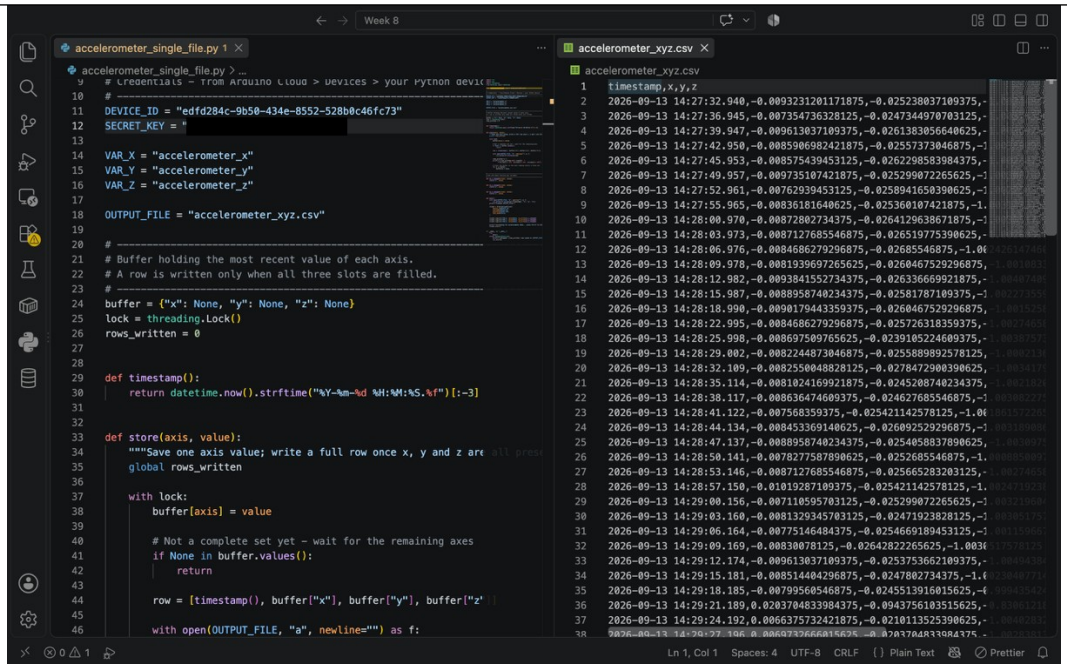

def on_z_changed(client, value):
    append_row(FILES[VAR_Z], value)
    print(f"z = {value:+.4f}")


def main():
    # Write a header row into each file before we start
    for name, filename in FILES.items():
        with open(filename, "w", newline="") as f:
            csv.writer(f).writerow(["timestamp", "value"])
        print(f"Created {filename}")

    client = ArduinoCloudClient(
        device_id=DEVICE_ID,
        username=DEVICE_ID,
        password=SECRET_KEY,
        sync_mode=False,
    )

    # Register each cloud variable with its own call-back
```

| | |
|---|---|
| | `client.register(VAR_X, value=None, on_write=on_x_changed)`<br>`client.register(VAR_Y, value=None, on_write=on_y_changed)`<br>`client.register(VAR_Z, value=None, on_write=on_z_changed)`<br><br>`print("\nListening for accelerometer data... press Ctrl+C to stop.\n")`<br>`client.start()`<br><br>`if __name__ == "__main__":`<br>`try:`<br>`main()`<br>`except KeyboardInterrupt:`<br>`print("\nStopped. CSV files saved.")`<br>`sys.exit(0)`<br><br>**accelerometer_x.csv (first 6 of 166 data rows)**<br>`timestamp,value`<br>`2026-09-13 14:14:21.566,-0.0054779052734375`<br>`2026-09-13 14:14:22.567,-0.0072784423828125`<br>`2026-09-13 14:14:25.571,-0.0081024169921875`<br>`2026-09-13 14:14:28.575,-0.00732421875`<br>`2026-09-13 14:14:31.580,-0.0072174072265625`<br>`2026-09-13 14:14:34.584,-0.0071868896484375`<br><br>**accelerometer_y.csv (first 6 of 167 data rows)**<br>`timestamp,value`<br>`2026-09-13 14:14:21.567,-0.0259246826171875`<br>`2026-09-13 14:14:23.569,-0.025543212890625`<br>`2026-09-13 14:14:26.572,-0.0252838134765625`<br>`2026-09-13 14:14:29.577,-0.026214599609375`<br>`2026-09-13 14:14:32.582,-0.02630615234375`<br>`2026-09-13 14:14:35.586,-0.02593994140625`<br><br>**accelerometer_z.csv (first 6 of 166 data rows)**<br>`timestamp,value`<br>`2026-09-13 14:14:21.567,-1.00421142578125`<br>`2026-09-13 14:14:24.570,-1.003204345703125`<br>`2026-09-13 14:14:27.574,-1.0021514892578125`<br>`2026-09-13 14:14:30.579,-1.0008544921875`<br>`2026-09-13 14:14:33.583,-1.0003814697265625`<br>`2026-09-13 14:14:36.587,-1.002197265625` |
| 4 | Question: Now manage 3 variable data so they can be stored in a single CSV file where each line consists of comma separated sensor values with a timestamp - <timestamp>, <x>, <y>, <z>. Store data once you gather 3 variables and repeat the process. Screenshot your Python script here and screenshot the file you have created opened and paste it here.<br><br>Answer: accelerometer_single_file.py keeps the latest x, y and z values in a buffer and writes one <timestamp>,<x>,<y>,<z> line to accelerometer_xyz.csv only once all three values have arrived, then clears the buffer and waits for the next set. This produced 95 rows, recorded on 13 September 2026 from 14:27:32 to 14:34:35. The screenshot shows the script next to the combined file. The full script and the first rows of the file are listed below it. |




## Python script: accelerometer_single_file.py

```
import csv
import sys
import threading
from datetime import datetime

from arduino_iot_cloud import ArduinoCloudClient

# ----------------------------------------------------------------------------
# Credentials - from Arduino Cloud > Devices > your Python device
# ----------------------------------------------------------------------------
DEVICE_ID = "edfd284c-9b50-434e-8552-528b0c46fc73"
SECRET_KEY = "<hidden for submission>"

VAR_X = "accelerometer_x"
VAR_Y = "accelerometer_y"
VAR_Z = "accelerometer_z"

OUTPUT_FILE = "accelerometer_xyz.csv"

# ----------------------------------------------------------------------------
# Buffer holding the most recent value of each axis.
# A row is written only when all three slots are filled.
# ----------------------------------------------------------------------------
buffer = {"x": None, "y": None, "z": None}
lock = threading.Lock()
rows_written = 0


def timestamp():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]


def store(axis, value):
    """Save one axis value; write a full row once x, y and z are all present."""
    global rows_written

    with lock:
        buffer[axis] = value

        # Not a complete set yet - wait for the remaining axes
        if None in buffer.values():
            return

        row = [timestamp(), buffer["x"], buffer["y"], buffer["z"]]

        with open(OUTPUT_FILE, "a", newline="") as f:
            csv.writer(f).writerow(row)

        rows_written += 1
        print(f"[{rows_written:4d}] {row[0]}  "
              f"x={row[1]:+.4f}  y={row[2]:+.4f}  z={row[3]:+.4f}")

        # Clear the buffer so the next reading starts a fresh set
        for k in buffer:
            buffer[k] = None


# ----------------------------------------------------------------------------
# One call-back function per variable
# ----------------------------------------------------------------------------
def on_x_changed(client, value):
    store("x", value)


def on_y_changed(client, value):
    store("y", value)


def on_z_changed(client, value):
```

```
    store("z", value)


def main():
    with open(OUTPUT_FILE, "w", newline="") as f:
        csv.writer(f).writerow(["timestamp", "x", "y", "z"])
    print(f"Created {OUTPUT_FILE}")

    client = ArduinoCloudClient(
        device_id=DEVICE_ID,
        username=DEVICE_ID,
        password=SECRET_KEY,
        sync_mode=False,
    )

    client.register(VAR_X, value=None, on_write=on_x_changed)
    client.register(VAR_Y, value=None, on_write=on_y_changed)
    client.register(VAR_Z, value=None, on_write=on_z_changed)

    print("\nListening for accelerometer data... press Ctrl+C to stop.\n")
    client.start()


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print(f"\nStopped. {rows_written} rows saved to {OUTPUT_FILE}.")
        sys.exit(0)
```

## accelerometer_xyz.csv (first 6 of 95 data rows)

```
timestamp,x,y,z
2026-09-13 14:27:32.940,-0.0093231201171875,-0.025238037109375,-1.0034484863281250
2026-09-13 14:27:36.945,-0.007354736328125,-0.0247344970703125,-1.003753662109375
2026-09-13 14:27:39.947,-0.009613037109375,-0.0261383056640625,-1.0036926269531250
2026-09-13 14:27:42.950,-0.0085906982421875,-0.02557373046875,-1.0038909912109375
2026-09-13 14:27:45.953,-0.008575439453125,-0.0262298583984375,-1.0024871826171875
2026-09-13 14:27:49.957,-0.009735107421875,-0.025299072265625,-1.0030670166015625
```

## Appendix C: Makerspace soldering induction certificate

